Front. Comput. Sci., 2025, 0(0): 1–42
https://doi.org/10.1007/s11704-024-40231-1

REVIEW ARTICLE

A Survey on Large Language Model based Autonomous Agents

Lei Wang, Chen Ma*, Xueyang Feng*, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhi-Yuan Chen, Jiakai Tang, Xu Chen(✉), Yankai Lin(✉), Wayne Xin Zhao, Zhewei Wei, Ji-Rong Wen

Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, 100872, China

© Higher Education Press 2025

arXiv:2308.11432v7 [cs.AI] 2 Mar 2025

Abstract Autonomous agents have long been a research focus in academic and industry communities. Previous research often focuses on training agents with limited knowledge within isolated environments, which diverges significantly from human learning processes, and makes the agents hard to achieve human-like decisions. Recently, through the acquisition of vast amounts of web knowledge, large language models (LLMs) have shown potential in human-level intelligence, leading to a surge in research on LLM-based autonomous agents. In this paper, we present a comprehensive survey of these studies, delivering a systematic review of LLM-based autonomous agents from a holistic perspective. We first discuss the construction of LLM-based autonomous agents, proposing a unified framework that encompasses much of previous work. Then, we present an overview of the diverse applications of LLM-based autonomous agents in social science, natural science, and engineering. Finally, we delve into the evaluation strategies commonly used for LLM-based autonomous agents. Based on the previous studies, we also present several challenges and future directions in this field.

Keywords Autonomous agent, Large language model, Human-level intelligence

1 Introduction

"An autonomous agent is a system situated within and a part of an environment that senses that environment and acts on it, over time, in pursuit of its own agenda and so as to effect what it senses in the future."

Franklin and Graesser (1997)

Autonomous agents have long been recognized as a promising approach to achieving artificial general intelligence (AGI), which is expected to ac-

Received month dd, yyyy; accepted month dd, yyyy

E-mail: xu.chen@ruc.edu.cn;yankailin@ruc.edu.cn
*Both authors contribute equally to this paper.

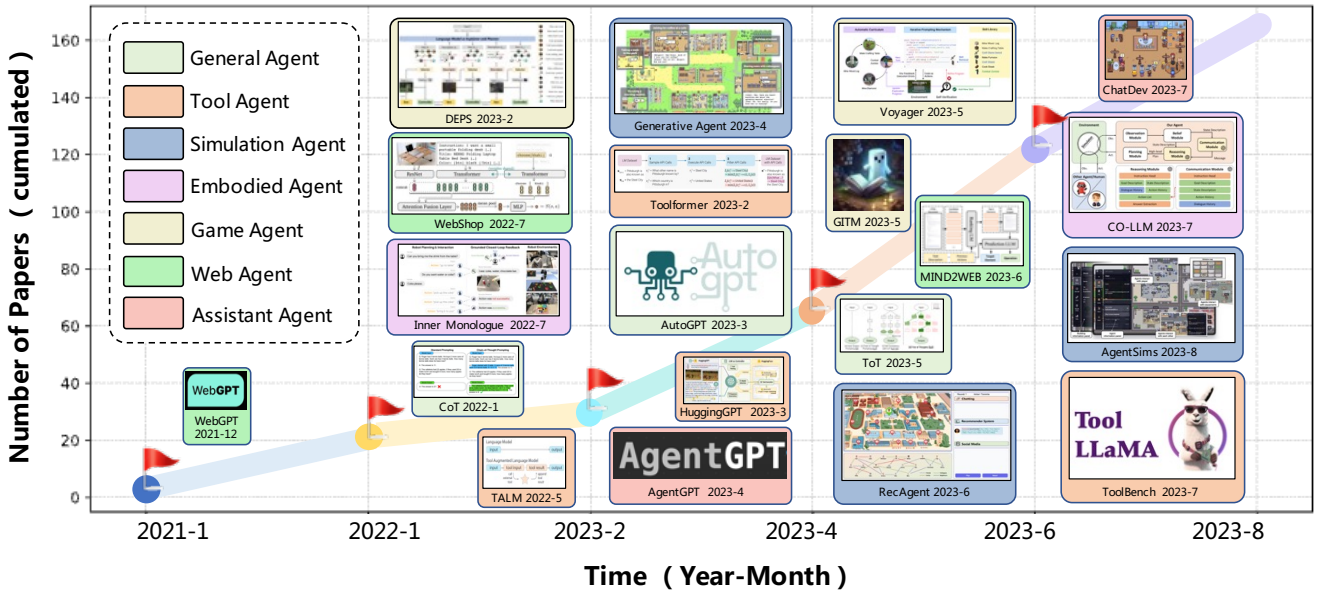

Fig. 1 Illustration of the growth trend in the field of LLM-based autonomous agents. We present the cumulative number of papers published from January 2021 to August 2023. We assign different colors to represent various agent categories. For example, a game agent aims to simulate a game-player, while a tool agent mainly focuses on tool using. For each time period, we provide a curated list of studies with diverse agent categories.

complish tasks through self-directed planning and actions. In previous studies, the agents are assumed to act based on simple and heuristic policy functions, and learned in isolated and restricted environments [1–6]. Such assumptions significantly differs from the human learning process, since the human mind is highly complex and individuals can learn from a much wider variety of environments. Because of these gaps, the agents obtained from previous studies are usually far from replicating human-level decision processes, especially in unconstrained, open-domain settings.

In recent years, large language models (LLMs) have achieved notable successes, demonstrating significant potential to achieve human-like intelligence [5–10]. This capability arises from leveraging comprehensive training datasets alongside a substantial number of model parameters. Building upon this capability, there has been a growing research area that employs LLMs as central controllers to construct autonomous agents to obtain human-like decision-making capabilities [11–17].

Compared to reinforcement learning, LLM-based agents possess more comprehensive internal world knowledge, enabling them to perform informed actions even without training on specific domain data. Furthermore, LLM-based agents can offer natural language interfaces for human interaction, providing greater flexibility and enhanced explainability.

Along this direction, researchers have developed numerous promising models (see Figure 1 for an overview), where the key idea is to equip LLMs with human capabilities such as memory and planning to make them behave like humans and complete various tasks effectively. Previously, these models were proposed independently, with limited efforts made to summarize and compare them holistically. However, we believe that a systematic sum-

mary of this rapidly developing field is of great significance for a comprehensive understanding of it and is beneficial in inspiring future research.

In this paper, we conduct a comprehensive survey of the field of LLM-based autonomous agents. We organize our survey around three key aspects: construction, application, and evaluation of LLM-based autonomous agents. For agent construction, we focus on two problems, that is, (1) how to design the agent architecture to better leverage LLMs, and (2) how to inspire and enhance the agent capability to complete different tasks. Intuitively, the first problem aims to build the hardware fundamentals for the agent, while the second problem focuses on providing the agent with software resources. For the first problem, we present a unified agent framework, which can encompass most of the previous studies. For the second problem, we provide a summary on the commonly-used strategies for agents' capability acquisition. In addition to discussing agent construction, we also provide a systematic overview of the applications of LLM-based autonomous agents in social science, natural science, and engineering. Finally, we delve into the strategies for evaluating LLM-based autonomous agents, focusing on both subjective and objective strategies.

In summary, this survey conducts a systematic review and establishes comprehensive taxonomies for existing studies in the burgeoning field of LLM-based autonomous agents. Our focus encompasses three primary areas: the construction of agents, their applications, and methods of evaluation. Drawing from a wealth of previous studies, we identify various challenges in this field and discuss potential future directions. We expect that our survey can provide newcomers of LLM-based autonomous agents with a comprehensive background knowledge, and also encourage further groundbreaking studies.

2 LLM-based Autonomous Agent Construction

LLM-based autonomous agents are expected to effectively perform diverse tasks by leveraging the human-like capabilities of LLMs. In order to achieve this goal, there are two significant aspects: (1) which architecture should be designed to better use LLMs and (2) given the designed architecture, how to enable the agent to acquire capabilities for accomplishing specific tasks. Within the context of architecture design, we contribute a systematic synthesis of existing research, culminating in a comprehensive unified framework. As for the second aspect, we summarize the strategies for agent capability acquisition based on whether they fine-tune the LLMs. Comparing LLM-based autonomous agents to traditional machine learning, architecture design is analogous to defining the network structure, while capability acquisition resembles the process of learning network parameters. In the following sections, we explore these two aspects in greater detail.

2.1 Agent Architecture Design

Recent advancements in LLMs have demonstrated their great potential to accomplish a wide range of tasks in the form of question-answering (QA). However, building autonomous agents is far from QA, since they need to fulfill specific roles and autonomously perceive and learn from the environment to evolve themselves like humans. To bridge the gap between traditional LLMs and autonomous agents, a crucial aspect is to design rational agent architectures to assist LLMs in maximizing their capabilities. Along this direction, previous work has developed a number of modules to enhance LLMs. In this section, we propose a unified framework to summarize these modules. Specifically, the overall

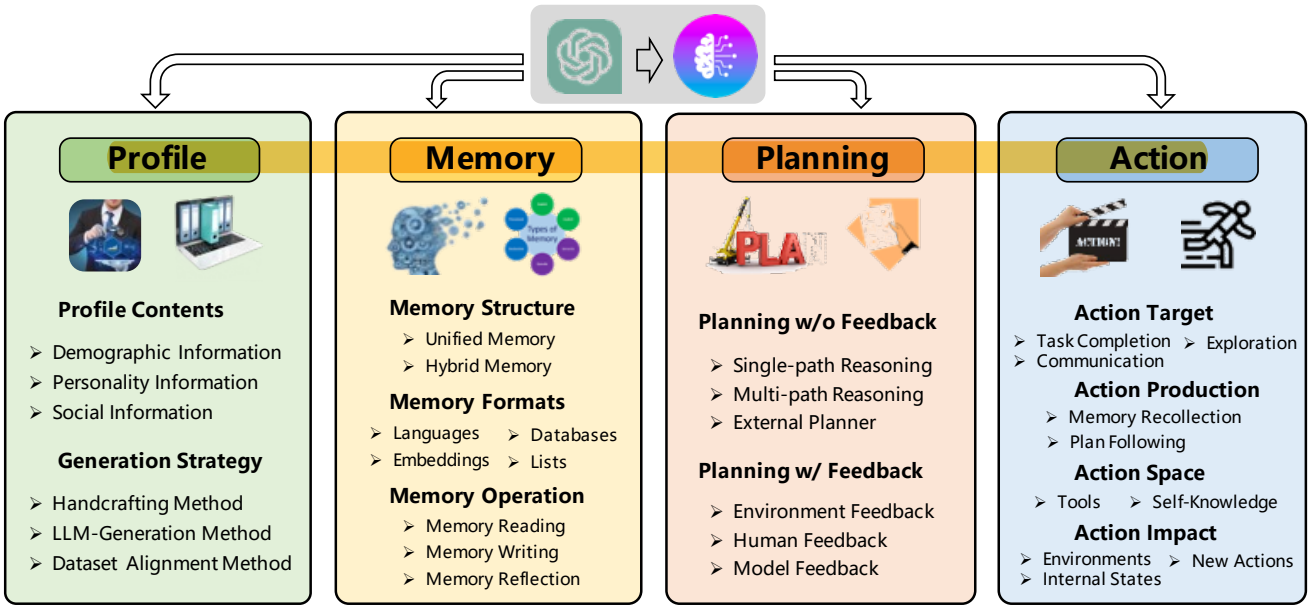

Fig. 2 A unified framework for the architecture design of LLM-based autonomous agent.

structure of our framework is illustrated in Figure 2, which is composed of a profiling module, a memory module, a planning module, and an action module. The purpose of the profiling module is to identify the role of the agent. The memory and planning modules place the agent into a dynamic environment, enabling it to recall past behaviors and plan future actions. The action module is responsible for translating the agent's decisions into specific outputs. Within these modules, the profiling module impacts the memory and planning modules, and collectively, these three modules influence the action module. In the following, we detail these modules.

2.1.1 Profiling Module

Autonomous agents typically perform tasks by assuming specific roles, such as coders, teachers, and domain experts [18, 19]. The profiling module aims to indicate the profiles of the agent roles, which are usually written into the prompt to influence the behavior of the LLM. Agent profiles typically encompass basic information such as age, gender, and career [20], as well as psychology information, reflecting the personalities of the agents, and social information, detailing the relationships between agents [21]. The choice of information to profile the agent is largely determined by the specific application scenarios. For instance, if the application aims to study human cognitive process, then the psychology information becomes pivotal. After identifying the types of profile information, the next important problem is to create specific profiles for the agents. Existing literature commonly employs the following three strategies.

Handcrafting Method: in this method, agent profiles are manually specified. For instance, if one would like to design agents with different personalities, he can use "you are an outgoing person" or "you are an introverted person" to profile the agent. The handcrafting method has been leveraged in a lot of previous work to specify the agent profiles. For example, Generative Agent [22] describes the agent by the information such as name, objectives, and relationships with other agents. MetaGPT [23], ChatDev [18], and Self-collaboration [24] predefine

various roles and their corresponding responsibilities in software development, manually assigning distinct profiles to each agent to facilitate collaboration. PTLLM [25] aims to explore and quantify personality traits displayed in texts generated by LLMs. This method guides LLMs in generating diverse responses by manually defining various agent characters through the use of personality assessment tools such as IPIP-NEO [26] and BFI [27]. [28] studies the toxicity of the LLM output by manually prompting LLMs with different roles, such as politicians, journalists and businesspersons. In general, the handcrafting method is very flexible, since one can assign any profile information to the agents. However, it can be also labor-intensive, particularly when dealing with a large number of agents.

LLM-generation Method: in this method, agent profiles are automatically generated based on LLMs. Typically, it begins by indicating the profile generation rules, elucidating the composition and attributes of the agent profiles within the target population. Then, one can optionally specify several seed agent profiles to serve as few-shot examples. Finally, LLMs are leveraged to generate all the agent profiles. For example, RecAgent [21] first creates seed profiles for a few agents by manually crafting their attributes such as age, gender, personal traits, and movie preferences. Then, it leverages ChatGPT to generate more agent profiles based on the seed information. This approach significantly reduces the time and effort required to construct agent profiles, particularly for large-scale populations. However, it may lack precise control over the generated profiles, which can result in inconsistencies or deviations from the intended characteristics.

Dataset Alignment Method: in this method, the agent profiles are obtained from real-world datasets. Typically, one can first organize the information about real humans in the datasets into natural language prompts, and then leverage it to profile the agents. For instance, in [29], the authors assign roles to GPT-3 based on the demographic backgrounds (such as race/ethnicity, gender, age, and state of residence) of participants in the American National Election Studies (ANES). They subsequently investigate whether GPT-3 can produce similar results to those of real humans. The dataset alignment method accurately captures the attributes of the real population, thereby making the agent behaviors more meaningful and reflective of real-world scenarios.

Remark. While most of the previous work leverage the above profile generation strategies independently, we argue that combining them may yield additional benefits. For example, in order to predict social developments via agent simulation, one can leverage real-world datasets to profile a subset of the agents, thereby accurately reflecting the current social status. Subsequently, roles that do not exist in the real world but may emerge in the future can be manually assigned to the other agents, enabling the prediction of future social development. Beyond this example, one can also flexibly combine the other strategies. The profile module serves as the foundation for agent design, exerting significant influence on the agent memorization, planning, and action procedures.

2.1.2 Memory Module

The memory module plays a very important role in the agent architecture design. It stores information perceived from the environment and leverages the recorded memories to facilitate future actions. The memory module can help the agent to accumulate experiences, self-evolve, and behave in a more consistent, reasonable, and effective manner. This

section provides a comprehensive overview of the memory module, focusing on its structures, formats, and operations.

Memory Structures: LLM-based autonomous agents often draw inspiration from cognitive science research on human memory processes. Human memory follows a general progression from sensory memory that registers perceptual inputs, to short-term memory that maintains information transiently, to long-term memory that consolidates information over extended periods. When designing the agent memory structures, researchers take inspiration from these aspects of human memory. In particular, short-term memory is analogous to the input information within the context window constrained by the transformer architecture. Long-term memory resembles the external vector storage that agents can rapidly query and retrieve from as needed. In the following, we introduce two commonly used memory structures based on the short-term and long-term memories.

• *Unified Memory.* This structure only simulates the human short-term memory, which is usually realized by in-context learning, and the memory information is directly written into the prompts. For example, RLP [30] is a conversation agent, which maintains internal states for the speaker and listener. During each round of conversation, these states serve as LLM prompts, functioning as the agent's short-term memory. SayPlan [31] is an embodied agent specifically designed for task planning. In this agent, the scene graphs and environment feedback serve as the agent's short-term memory, guiding its actions. CALYPSO [32] is an agent designed for the game Dungeons & Dragons, which can assist Dungeon Masters in the creation and narration of stories. Its short-term memory is built upon scene descriptions, monster information, and previous summaries. DEPS [33] is also a game agent, developed for Minecraft. The agent initially generates task plans and then utilizes them to prompt LLMs, which in turn produce actions to complete the task. These plans can be deemed as the agent's short-term memory. In practice, implementing short-term memory is straightforward and can enhance an agent's ability to perceive recent or contextually sensitive behaviors and observations. However, the limited context window of LLMs restricts incorporating comprehensive memories into prompts, which can impair agent performance. This challenge necessitates LLMs with larger context windows and the ability to handle extended contexts. Consequently, numerous researchers turn to hybrid memory systems to mitigate this issue.

• *Hybrid Memory.* This structure explicitly models the human short-term and long-term memories. The short-term memory temporarily buffers recent perceptions, while long-term memory consolidates important information over time. For instance, Generative Agent [20] employs a hybrid memory structure to facilitate agent behaviors. The short-term memory contains the context information about the agent current situations, while the long-term memory stores the agent past behaviors and thoughts, which can be retrieved according to the current events. AgentSims [34] also implements a hybrid memory architecture. The information provided in the prompt can be considered as short-term memory. In order to enhance the storage capacity of memory, the authors propose a long-term memory system that utilizes a vector database, facilitating efficient storage and retrieval. Specifically, the agent's daily memories are encoded as embeddings and stored in the vector database. If the agent needs to recall its previous memories, the long-term memory system retrieves relevant infor-

mation using embedding similarities. This process can improve the consistency of the agent's behavior. In GITM [16], the short-term memory stores the current trajectory, and the long-term memory saves reference plans summarized from successful prior trajectories. Long-term memory provides stable knowledge, while short-term memory allows flexible planning. Reflexion [12] utilizes a short-term sliding window to capture recent feedback and incorporates persistent long-term storage to retain condensed insights. This combination allows for the utilization of both detailed immediate experiences and high-level abstractions. SCM [35] selectively activates the most relevant long-term knowledge to combine with short-term memory, enabling reasoning over complex contextual dialogues. SimplyRetrieve [36] utilizes user queries as short-term memory and stores long-term memory using private knowledge bases. This design enhances the model accuracy while guaranteeing user privacy. MemorySandbox [37] implements long-term and short-term memory to store different objects, which can then be accessed throughout various conversations. Users can create multiple conversations with different agents on the same canvas, facilitating the sharing of memory objects through a simple drag-and-drop interface. In practice, integrating both short-term and long-term memories can enhance an agent's ability for long-range reasoning and accumulation of valuable experiences, which are crucial for accomplishing tasks in complex environments.

Remark. Careful readers may find that there may also exist another type of memory structure, that is, only based on the long-term memory. However, we find that such type of memory is rarely documented in the literature. Our speculation is that the agents are always situated in continuous and dynamic environments, with consecutive actions displaying a high correlation. Therefore, the capture of short-term memory is very important and usually cannot be disregarded.

Memory Formats: In addition to the memory structure, another perspective to analyze the memory module is based on the formats of the memory storage medium, for example, natural language memory or embedding memory. Different memory formats possess distinct strengths and are suitable for various applications. In the following, we introduce several representative memory formats.

• *Natural Languages.* In this format, memory information such as the agent behaviors and observations are directly described using raw natural language. This format possesses several strengths. Firstly, the memory information can be expressed in a flexible and understandable manner. Moreover, it retains rich semantic information that can provide comprehensive signals to guide agent behaviors. In the previous work, Reflexion [12] stores experiential feedback in natural language within a sliding window. Voyager [38] employs natural language descriptions to represent skills within the Minecraft game, which are directly stored in memory.

• *Embeddings.* In this format, memory information is encoded into embedding vectors, which enhances both retrieval and reading efficiency. For instance, MemoryBank [39] encodes each memory segment as an embedding vector and employs a dual-tower dense retrieval model to efficiently retrieve relevant information from past conversations.

• *Databases.* In this format, memory information is stored in databases, allowing the agent to manipulate memories efficiently and comprehensively. For example, ChatDB [40] uses a database as a symbolic memory module. The agent can utilize SQL statements to precisely add, delete, and modify the memory information.

• *Structured Lists*. In this format, memory information is organized into lists, and the semantic of memory can be conveyed in an efficient and concise manner. For instance, GITM [16] stores action lists for sub-goals in a hierarchical tree structure. The hierarchical structure explicitly captures the relationships between goals and corresponding plans. RET-LLM [41] initially converts natural language sentences into triplet phrases, and subsequently stores them in memory.

Remark. Here we only show several representative memory formats, but it is important to note that there are many uncovered ones, such as the programming code used by [38]. Moreover, it should be emphasized that these formats are not mutually exclusive; many models incorporate multiple formats to concurrently harness their respective benefits. A notable example is the memory module of GITM [16], which utilizes a key-value list structure. In this structure, the keys are represented by embedding vectors, while the values consist of raw natural languages. The use of embedding vectors allows for efficient retrieval of memory records. By utilizing natural languages, the memory contents become highly comprehensive, enabling more informed agent actions.

Above, we mainly discuss the internal designs of the memory module. In the following, we turn our focus to memory operations, which are used to interact with external environments.

Memory Operations: The memory module plays a critical role in allowing the agent to acquire, accumulate, and utilize significant knowledge by interacting with the environment. The interaction between the agent and the environment is accomplished through three crucial memory operations: memory reading, memory writing, and memory reflection. In the following, we introduce these operations more in detail.

• *Memory Reading*. The objective of memory reading is to extract meaningful information from memory to enhance the agent's actions. For example, using the previously successful actions to achieve similar goals [16]. The key of memory reading lies in how to extract valuable information from history actions. Usually, there are three commonly used criteria for information extraction, that is, the recency, relevance, and importance [20]. Memories that are more recent, relevant, and important are more likely to be extracted. Formally, we conclude the following equation from existing literature for memory information extraction:

$$m^* = \arg\max_{m \in M} \left(\alpha s^{rec}(q, m) + \beta s^{rel}(q, m) + \gamma s^{imp}(m) \right), \tag{1}$$

where q is the query, for example, the task that the agent should address or the context in which the agent is situated. M is the set of all memories. $s^{rec}(\cdot)$, $s^{rel}(\cdot)$ and $s^{imp}(\cdot)$ are the scoring functions for measuring the recency, relevance, and importance of the memory m, with higher scores indicating more recent, more relevant, and more important memories respectively. These scoring functions can be implemented using various methods, for example, $s^{rel}(q, m)$ can be calculated using vector similarity measures between query and memory embeddings. It should be noted that s^{imp} only reflects the characters of the memory itself, thus it is unrelated to the query q. α, β and γ are balancing parameters. By assigning them with different values, one can obtain various memory reading strategies. For example, by setting $\alpha = \gamma = 0$, many studies [16, 30, 38, 41] only consider the relevance score s^{rel} for memory reading. By assigning $\alpha = \beta = \gamma = 1.0$, [20] equally weights all the above three metrics to extract information from memory.

• *Memory Writing*. The purpose of memory

writing is to store information about the perceived environment in memory. Storing valuable information in memory provides a foundation for retrieving informative memories in the future, enabling the agent to act more efficiently and rationally. During the memory writing process, there are two potential problems that should be carefully addressed. On one hand, it is crucial to address how to store information that is similar to existing memories (*i.e.*, memory duplicated). On the other hand, it is important to consider how to remove information when the memory reaches its storage limit (*i.e.*, memory overflow). In the following, we discuss these problems more in detail. (1) *Memory Duplicated*. To incorporate similar information, people have developed various methods for integrating new and previous records. For instance, in [16], the successful action sequences related to the same subgoal are stored in a list. Once the size of the list reaches N(=5), all the sequences in it are condensed into a unified plan solution using LLMs. The original sequences in the memory are replaced with the newly generated one. Augmented LLM [42] aggregates duplicate information via count accumulation, avoiding redundant storage. (2) *Memory Overflow*. In order to write information into the memory when it is full, people design different methods to delete existing information to continue the memorizing process. For example, in ChatDB [40], memories can be explicitly deleted based on user commands. RET-LLM [41] uses a fixed-size buffer for memory, overwriting the oldest entries in a first-in-first-out (FIFO) manner.

• *Memory Reflection*. Memory reflection emulates humans' ability to witness and evaluate their own cognitive, emotional, and behavioral processes. When adapted to agents, the objective is to provide agents with the capability to independently summarize and infer more abstract, complex and high-level information. More specifically, in Generative Agent [20], the agent has the capability to summarize its past experiences stored in memory into broader and more abstract insights. To begin with, the agent generates three key questions based on its recent memories. Then, these questions are used to query the memory to obtain relevant information. Building upon the acquired information, the agent generates five insights, which reflect the agent high-level ideas. For example, the low-level memories "Klaus Mueller is writing a research paper", "Klaus Mueller is engaging with a librarian to further his research", and "Klaus Mueller is conversing with Ayesha Khan about his research" can induce the high-level insight "Klaus Mueller is dedicated to his research". In addition, the reflection process can occur hierarchically, meaning that the insights can be generated based on existing insights. In GITM [16], the actions that successfully accomplish the sub-goals are stored in a list. When the list contains more than five elements, the agent summarizes them into a common and abstract pattern and replaces all the elements. In ExpeL [43], two approaches are introduced for the agent to acquire reflection. Firstly, the agent compares successful or failed trajectories within the same task. Secondly, the agent learns from a collection of successful trajectories to gain experiences.

A significant distinction between traditional LLMs and the agents is that the latter must possess the capability to learn and complete tasks in dynamic environments. If we consider the memory module as responsible for managing the agents' past behaviors, it becomes essential to have another significant module that can assist the agents in planning their future actions. In the following, we present an overview of how researchers design the planning module.

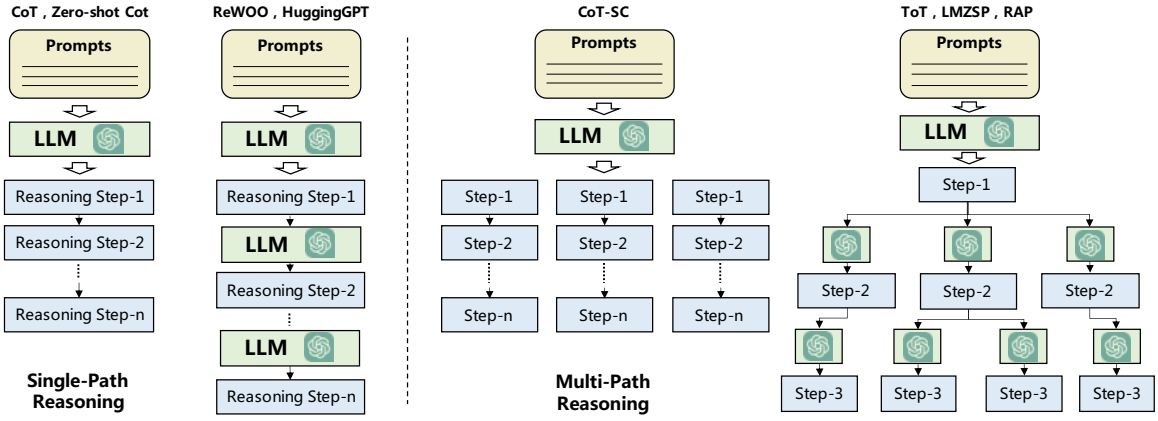

Fig. 3 Comparison between the strategies of single-path and multi-path reasoning. LMZSP is the model proposed in [44].

2.1.3 Planning Module

When faced with a complex task, humans tend to deconstruct it into simpler subtasks and solve them individually. The planning module aims to empower the agents with such human capability, which is expected to make the agent behave more reasonably, powerfully, and reliably. In specific, we summarize existing studies based on whether the agent can receive feedback in the planing process, which are detailed as follows:

Planning without Feedback: In this method, the agents do not receive feedback that can influence its future behaviors after taking actions. In the following, we present several representative strategies.

• *Single-path Reasoning*. In this strategy, the final task is decomposed into several intermediate steps. These steps are connected in a cascading manner, with each step leading to only one subsequent step. LLMs follow these steps to achieve the final goal. Specifically, Chain of Thought (CoT) [45] proposes inputting reasoning steps for solving complex problems into the prompt. These steps serve as examples to inspire LLMs to plan and act in a step-by-step manner. In this method, the plans are created based on the inspiration from the examples in the prompts. Zero-shot-CoT [46] enables LLMs to generate task reasoning processes by prompting them with trigger sentences like "think step by step". Unlike CoT, this method does not incorporate reasoning steps as examples in the prompts. Re-Prompting [47] involves checking whether each step meets the necessary prerequisites before generating a plan. If a step fails to meet the prerequisites, it introduces a prerequisite error message and prompts the LLM to regenerate the plan. ReWOO [48] introduces a paradigm of separating plans from external observations, where the agents first generate plans and obtain observations independently, and then combine them together to derive the final results. HuggingGPT [13] first decomposes the task into many sub-goals, and then solves each of them based on Huggingface. Different from CoT and Zero-shot-CoT, which outcome all the reasoning steps in a one-shot manner, ReWOO and HuggingGPT produce the results by accessing LLMs multiply times. SWIFTSAGE [49], inspired by the dual-process theory of human cognition [50], combines the power of both SWIFT and SAGE modules for planning in complex interactive tasks. The SWIFT module provides quick responses based on learned patterns, while the SAGE module, using large language models, conducts in-depth planning by ask-

ing key questions and generating action sequences to ensure successful task completion.

• *Multi-path Reasoning*. In this strategy, the reasoning steps for generating the final plans are organized into a tree-like structure. Each intermediate step may have multiple subsequent steps. This approach is analogous to human thinking, as individuals may have multiple choices at each reasoning step. In specific, Self-consistent CoT (CoT-SC) [51] believes that each complex problem has multiple ways of thinking to deduce the final answer. Thus, it starts by employing CoT to generate various reasoning paths and corresponding answers. Subsequently, the answer with the highest frequency is chosen as the final output. Tree of Thoughts (ToT) [52] is designed to generate plans using a tree-like reasoning structure. In this approach, each node in the tree represents a "thought," which corresponds to an intermediate reasoning step. The selection of these intermediate steps is based on the evaluation of LLMs. The final plan is generated using either the breadth-first search (BFS) or depth-first search (DFS) strategy. Comparing with CoT-SC, which generates all the planed steps together, ToT needs to query LLMs for each reasoning step. In RecMind [53], the authors designed a self-inspiring mechanism, where the discarded historical information in the planning process is also leveraged to derive new reasoning steps. In GoT [54], the authors expand the tree-like reasoning structure in ToT to graph structures, resulting in more powerful prompting strategies. In AoT [55], the authors design a novel method to enhance the reasoning processes of LLMs by incorporating algorithmic examples into the prompts. Remarkably, this method only needs to query LLMs for only one or a few times. In [44], the LLMs are leveraged as zero-shot planners. At each planning step, they first generate multiple possible next steps, and then determine the final one based on their distances to admissible actions. [56] further improves [44] by incorporating examples that are similar to the queries in the prompts. RAP [57] builds a world model to simulate the potential benefits of different plans based on Monte Carlo Tree Search (MCTS), and then, the final plan is generated by aggregating multiple MCTS iterations. To enhance comprehension, we provide an illustration comparing the strategies of single-path and multi-path reasoning in Figure 3.

• *External Planner*. Despite the demonstrated power of LLMs in zero-shot planning, effectively generating plans for domain-specific problems remains highly challenging. To address this challenge, researchers turn to external planners. These tools are well-developed and employ efficient search algorithms to rapidly identify correct, or even optimal, plans. In specific, LLM+P [58] first transforms the task descriptions into formal Planning Domain Definition Languages (PDDL), and then it uses an external planner to deal with the PDDL. Finally, the generated results are transformed back into natural language by LLMs. Similarly, LLM-DP [59] utilizes LLMs to convert the observations, the current world state, and the target objectives into PDDL. Subsequently, this transformed data is passed to an external planner, which efficiently determines the final action sequence. CO-LLM [22] demonstrates that LLMs is good at generating high-level plans, but struggle with low-level control. To address this limitation, a heuristically designed external low-level planner is employed to effectively execute actions based on high-level plans.

Planning with Feedback: In many real-world scenarios, the agents need to make long-horizon planning to solve complex tasks. When facing these tasks, the above planning modules without feedback

can be less effective due to the following reasons: firstly, generating a flawless plan directly from the beginning is extremely difficult as it needs to consider various complex preconditions. As a result, simply following the initial plan often leads to failure. Moreover, the execution of the plan may be hindered by unpredictable transition dynamics, rendering the initial plan non-executable. Simultaneously, when examining how humans tackle complex tasks, we find that individuals may iteratively make and revise their plans based on external feedback. To simulate such human capability, researchers have designed many planning modules, where the agent can receive feedback after taking actions. The feedback can be obtained from environments, humans, and models, which are detailed in the following.

• *Environmental Feedback*. This feedback is obtained from the objective world or virtual environment. For instance, it could be the game's task completion signals or the observations made after the agent takes an action. In specific, ReAct [60] proposes constructing prompts using thought-act-observation triplets. The thought component aims to facilitate high-level reasoning and planning for guiding agent behaviors. The act represents a specific action taken by the agent. The observation corresponds to the outcome of the action, acquired through external feedback, such as search engine results. The next thought is influenced by the previous observations, which makes the generated plans more adaptive to the environment. Voyager [38] makes plans by incorporating three types of environment feedback including the intermediate progress of program execution, the execution error and self-verification results. These signals can help the agent to make better plans for the next action. Similar to Voyager, Ghost [16] also incorporates feedback into the reasoning and action taking processes. This feedback encompasses the environment states as well as the success and failure information for each executed action. SayPlan [31] leverages environmental feedback derived from a scene graph simulator to validate and refine its strategic formulations. This simulator is adept at discerning the outcomes and state transitions subsequent to agent actions, facilitating SayPlan's iterative recalibration of its strategies until a viable plan is ascertained. In DEPS [33], the authors argue that solely providing information about the completion of a task is often inadequate for correcting planning errors. Therefore, they propose informing the agent about the detail reasons for task failure, allowing them to more effectively revise their plans. LLM-Planner [61] introduces a grounded re-planning algorithm that dynamically updates plans generated by LLMs when encountering object mismatches and unattainable plans during task completion. Inner Monologue [62] provides three types of feedback to the agent after it takes actions: (1) whether the task is successfully completed, (2) passive scene descriptions, and (3) active scene descriptions. The former two are generated from the environments, which makes the agent actions more reasonable.

• *Human Feedback*. In addition to obtaining feedback from the environment, directly interacting with humans is also a very intuitive strategy to enhance the agent planning capability. The human feedback is a subjective signal. It can effectively make the agent align with the human values and preferences, and also help to alleviate the hallucination problem. In Inner Monologue [62], the agent aims to perform high-level natural language instructions in a 3D visual environment. It is given the capability to actively solicit feedback from humans regarding scene descriptions. Then, the agent incorporates the human feedback into its prompts,

enabling more informed planning and reasoning. In the above cases, we can see, different types of feedback can be combined to enhance the agent planning capability. For example, Inner Monologue [62] collects both environment and human feedback to facilitate the agent plans.

• *Model Feedback*. Apart from the aforementioned environmental and human feedback, which are external signals, researchers have also investigated the utilization of internal feedback from the agents themselves. This type of feedback is usually generated based on pre-trained models. In specific, [63] proposes a self-refine mechanism. This mechanism consists of three crucial components: output, feedback, and refinement. Firstly, the agent generates an output. Then, it utilizes LLMs to provide feedback on the output and offer guidance on how to refine it. At last, the output is improved by the feedback and refinement. This output-feedback-refinement process iterates until reaching some desired conditions. SelfCheck [64] allows agents to examine and evaluate their reasoning steps generated at various stages. They can then correct any errors by comparing the outcomes. InterAct [65] uses different language models (such as ChatGPT and InstructGPT) as auxiliary roles, such as checkers and sorters, to help the main language model avoid erroneous and inefficient actions. ChatCoT [66] utilizes model feedback to improve the quality of its reasoning process. The model feedback is generated by an evaluation module that monitors the agent reasoning steps. Reflexion [12] is developed to enhance the agent's planning capability through detailed verbal feedback. In this model, the agent first produces an action based on its memory, and then, the evaluator generates feedback by taking the agent trajectory as input. In contrast to previous studies, where the feedback is given as a scalar value, this model leverages LLMs to provide more detailed verbal feedback, which can provide more comprehensive supports for the agent plans.

Remark. In conclusion, the implementation of planning module without feedback is relatively straightforward. However, it is primarily suitable for simple tasks that only require a small number of reasoning steps. Conversely, the strategy of planning with feedback needs more careful designs to handle the feedback. Nevertheless, it is considerably more powerful and capable of effectively addressing complex tasks that involve long-range reasoning.

2.1.4 Action Module

The action module is responsible for translating the agent's decisions into specific outcomes. This module is located at the most downstream position and directly interacts with the environment. It is influenced by the profile, memory, and planning modules. This section introduces the action module from four perspectives: (1) Action goal: what are the intended outcomes of the actions? (2) Action production: how are the actions generated? (3) Action space: what are the available actions? (4) Action impact: what are the consequences of the actions? Among these perspectives, the first two focus on the aspects preceding the action ("before-action" aspects), the third focuses on the action itself ("in-action" aspect), and the fourth emphasizes the impact of the actions ("after-action" aspect).

Action Goal: The agent can perform actions with various objectives. Here, we present several representative examples: (1) *Task Completion*. In this scenario, the agent's actions are aimed at accomplishing specific tasks, such as crafting an iron pickaxe in Minecraft [38] or completing a function in software development [18]. These actions usually have well-defined objectives, and each action con-

tributes to the completion of the final task. Actions aimed at this type of goal are very common in existing literature. (2) *Communication*. In this case, the actions are taken to communicate with the other agents or real humans for sharing information or collaboration. For example, the agents in ChatDev [18] may communicate with each other to collectively accomplish software development tasks. In Inner Monologue [62], the agent actively engages in communication with humans and adjusts its action strategies based on human feedback. (3) *Environment Exploration*. In this example, the agent aims to explore unfamiliar environments to expand its perception and strike a balance between exploring and exploiting. For instance, the agent in Voyager [38] may explore unknown skills in their task completion process and continually refine the skill execution code based on environment feedback through trial and error.

Action Production: Different from ordinary LLMs, where the model input and output are directly associated, the agent may take actions via different strategies and sources. In the following, we introduce two types of commonly used action production strategies. (1) *Action via Memory Recollection*. In this strategy, the action is generated by extracting information from the agent memory according to the current task. The task and the extracted memories are used as prompts to trigger the agent actions. For example, in Generative Agents [20], the agent maintains a memory stream, and before taking each action, it retrieves recent, relevant and important information from the memory steam to guide the agent actions. In GITM [16], in order to achieve a low-level sub-goal, the agent queries its memory to determine if there are any successful experiences related to the task. If similar tasks have been completed previously, the agent invokes the previously successful actions to handle the current task directly. In collaborative agents such as ChatDev [18] and MetaGPT [23], different agents may communicate with each other. In this process, the conversation history in a dialog is remembered in the agent memories. Each utterance generated by the agent is influenced by its memory. (2) *Action via Plan Following*. In this strategy, the agent takes actions following its pre-generated plans. For instance, in DEPS [33], for a given task, the agent first makes action plans. If there are no signals indicating plan failure, the agent will strictly adhere to these plans. In GITM [16], the agent makes high-level plans by decomposing the task into many sub-goals. Based on these plans, the agent takes actions to solve each sub-goal sequentially to complete the final task.

Action Space: Action space refers to the set of possible actions that can be performed by the agent. In general, we can roughly divide these actions into two classes: (1) external tools and (2) internal knowledge of the LLMs. In the following, we introduce these actions more in detail.

- *External Tools*. While LLMs have been demonstrated to be effective in accomplishing a large amount of tasks, they may not work well for the domains which need comprehensive expert knowledge. In addition, LLMs may also encounter hallucination problems, which are hard to be resolved by themselves. To alleviate the above problems, the agents are empowered with the capability to call external tools for executing action. In the following, we present several representative tools which have been exploited in the literature.

(1) *APIs*. Leveraging external APIs to complement and expand action space is a popular paradigm in recent years. For example, HuggingGPT [13] integrates HuggingFace's vast model ecosystem to tackle complex user tasks. Similarly, WebGPT [67]

proposes to automatically generate queries to extract relevant content from external web pages when responding to user request. TPTU [68] explores the potential of LLMs to address intricate tasks through strategic task planning and API-based tools. Gorilla [69] introduces a fine-tuned LLM capable of generating precise input arguments for API calls, effectively mitigating hallucination issues during external API usage. ToolFormer [15] employs self-supervised learning to determine when and how to invoke external tools, using demonstrations of tool APIs for training. API-Bank [70] offers a comprehensive benchmark with a diverse collection of API tools to systematically evaluate tool-augmented LLMs, alongside robust training datasets designed to enhance their integration capabilities. ToolLLaMA [14] proposes a tool-use framework encompassing data collection, training, and evaluation, with the resulting fine-tuned model excelling across a wide array of APIs. RestGPT [71] connects LLMs with RESTful APIs, which follow widely accepted standards for web services development, making the resulting program more compatible with real-world applications. TaskMatrix.AI [72] connects LLMs with an extensive ecosystem of APIs to support task execution. At its core lies a multimodal conversational foundational model that interacts with users, understands their goals and context, and then produces executable code for particular tasks. In essence, these intelligent agents strategically harness external APIs as versatile tools, systematically expanding their action space and transcending the inherent limitations of traditional language models by integrating diverse computational capabilities.

(2) *Databases & Knowledge Bases*. Integrating external database or knowledge base enables agents to obtain specific domain information for generating more realistic actions. For example, ChatDB [40] employs SQL statements to query databases, facilitating actions by the agents in a logical manner. MRKL [73] and OpenAGI [74] incorporate various expert systems such as knowledge bases and planners to access domain-specific information.

(3) *External Models*. Previous studies often utilize external models to expand the range of possible actions. In comparison to APIs, external models typically handle more complex tasks. Each external model may correspond to multiple APIs. For example, ViperGPT [75] firstly uses Codex, which is implemented based on language model, to generate Python code from text descriptions, and then executes the code to complete the given tasks. ChemCrow [76] is an LLM-based chemical agent designed to perform tasks in organic synthesis, drug discovery, and material design. It utilizes seventeen expert-designed models to assist its operations. MM-REACT [77] integrates various external models, such as VideoBERT for video summarization, X-decoder for image generation, and SpeechBERT for audio processing, enhancing its capability in diverse multimodal scenarios.

• *Internal Knowledge*. In addition to utilizing external tools, many agents rely solely on the internal knowledge of LLMs to guide their actions. We now present several crucial capabilities of LLMs that can support the agent to behave reasonably and effectively. (1) *Planning Capability*. Previous work has demonstrated that LLMs can be used as decent planners to decompose complex tasks into simpler ones [45]. Such a capability of LLMs can be even triggered without incorporating examples in the prompts [46]. Building on the planning capability of LLMs, DEPS [33] develops a Minecraft agent, which can solve complex tasks via sub-goal decomposition. Similar agents like GITM [16] and Voyager [38] also heavily rely on the planning ca-

pability of LLMs to successfully complete various tasks. (2) *Conversation Capability.* LLMs can usually generate high-quality conversations. This capability enables agents to behave more like humans. In the previous work, many agents take actions based on the strong conversation capability of LLMs. For example, in ChatDev [18], different agents can discuss the software development process and reflect on their own behaviors. In RLP [30], the agent can communicate with the listeners based on their potential feedback on the agent's utterance. (3) *Common Sense Understanding Capability.* Another important capability of LLMs is that they can well comprehend human common sense. Based on this capability, many agents can simulate human daily life and make human-like decisions. For example, in Generative Agent [20], the agent can accurately understand its current state, the surrounding environment, and summarize high-level ideas based on basic observations. Without the common sense understanding capability of LLMs, these behaviors cannot be reliably simulated. Similar conclusions may also apply to RecAgent [21] and S3 [78], where the agents focus on simulating user social behaviors.

Action Impact: Action impact refers to the consequences of an agent's actions. While the range of possible impacts is vast, we highlight a few key examples for clarity: (1) *Changing Environments.* Agents can directly alter environment states by actions, such as moving their positions, collecting items, constructing buildings, etc. For instance, in GITM [16] and Voyager [38], the environments are changed by the actions of the agents in their task completion process. Specifically, when an agent collects resources—such as harvesting three pieces of wood—the resources disappear from the environment. (2) *Altering Internal States.* Actions taken by the agent can also change the agent itself, including updating memories, forming new plans, acquiring novel knowledge, and more. For example, in Generative Agents [20], memory streams are updated after performing actions within the system. Similarly, SayCan [79] enables agents to take actions to update understandings of the environment. (3) *Triggering New Actions.* In task completion processes, one action often leads to subsequent actions. For example, in Voyager [38], once the agent has gathered the necessary resources, it triggers the construction of buildings.

2.2 Agent Capability Acquisition

In the sections above, we focus mainly on how to design the agent architecture to better harness the capabilities of LLMs to enabling them to accomplish tasks akin to human performance. The architecture functions as the "hardware" of an agent. However, relying solely on the hardware is insufficient for achieving effective task performance. This is because the agent may lack the necessary task-specific capabilities, skills, and experiences, which can be regarded as "software" resources. In order to equip the agent with these resources, various strategies have been devised. Generally, we categorize these strategies into two classes based on whether they require fine-tuning of the LLMs. Below, we introduce each category in detail.

Capability Acquisition with Fine-tuning: A direct approach to enhance agent capabilities for task completion is to fine-tune the model using task-specific datasets. These datasets can be constructed from human annotations, LLM-generated content, or real-world applications. We discuss these methods in detail below.

• *Fine-tuning with Human Annotated Datasets.* To fine-tune the agent, utilizing human annotated datasets is a versatile approach that can be employed

Table 1 For the profile module, we use ①, ② and ③ to represent the handcrafting method, LLM-generation method, and dataset alignment method, respectively. For the memory module, we focus on the implementation strategies for memory operation and memory structure. For memory operation, we use ① and ② to indicate that the model only has read/write operations and has read/write/reflection operations, respectively. For memory structure, we use ① and ② to represent unified and hybrid memories, respectively. For the planning module, we use ① and ② to represent planning w/o feedback and w/ feedback, respectively. For the action module, we use ① and ② to represent that the model does not use tools and use tools, respectively. For the agent capability acquisition (CA) strategy, we use ① and ② to represent the methods with and without fine-tuning, respectively. "-" indicates that the corresponding content is not explicitly discussed in the paper.

Model	Profile	Memory		Planning	Action	CA	Time
		Operation	Structure				
WebGPT [67]	-	-	-	-	②	①	12/2021
SayCan [79]	-	-	-	①	①	②	04/2022
MRKL [73]	-	-	-	①	②	-	05/2022
Inner Monologue [62]	-	-	-	②	①	②	07/2022
Social Simulacra [80]	②	-	-	-	①	-	08/2022
ReAct [60]	-	-	-	②	②	①	10/2022
MALLM [42]	-	①	②	-	①	-	01/2023
DEPS [33]	-	-	-	②	①	②	02/2023
Toolformer [15]	-	-	-	①	②	①	02/2023
Reflexion [12]	-	②	②	②	①	②	03/2023
CAMEL [81]	① ②	-	-	②	①	-	03/2023
API-Bank [70]	-	-	-	②	②	②	04/2023
ViperGPT [75]	-	-	-	-	②	-	03/2023
HuggingGPT [13]	-	①	①	①	②	-	03/2023
Generative Agents [20]	①	②	②	②	①	-	04/2023
LLM+P [58]	-	-	-	①	①	-	04/2023
ChemCrow [76]	-	-	-	②	②	-	04/2023
OpenAGI [74]	-	-	-	②	②	①	04/2023
AutoGPT [82]	-	①	②	②	②	②	04/2023
SCM [35]	-	②	②	-	①	-	04/2023
Socially Alignment [83]	-	①	②	-	①	①	05/2023
GITM [16]	-	②	②	②	①	②	05/2023
Voyager [38]	-	②	②	②	①	②	05/2023
Introspective Tips [84]	-	-	-	②	①	②	05/2023
RET-LLM [41]	-	①	②	-	①	①	05/2023
ChatDB [40]	-	①	②	②	②	-	06/2023
S^3 [78]	③	②	②	-	①	-	07/2023
ChatDev [18]	①	②	②	②	①	②	07/2023
ToolLLM [14]	-	-	-	②	②	①	07/2023
MemoryBank [39]	-	②	②	-	①	-	07/2023
MetaGPT [23]	①	②	②	②	②	-	08/2023

in various application scenarios. In this approach, researchers first design annotation tasks and then recruit workers to complete them. For example, in CoH [85], the authors aim to align LLMs with human values and preferences. Different from the other models, where the human feedback is leveraged in a simple and symbolic manner, this method converts the human feedback into detailed comparison information in the form of natural languages. The LLMs are directly fine-tuned based on these natural language datasets. In RET-LLM [41], in order to better convert natural languages into structured memory information, the authors fine-tune LLMs based on a human constructed dataset, where each sample is a "triplet-natural language" pair. In WebShop [86], the authors collect 1.18 million real-world products from amazon.com, and put them onto a simulated e-commerce website, which con-

tains several carefully designed human shopping scenarios. Based on this website, the authors recruit 13 workers to collect a real-human behavior dataset. At last, three methods based on heuristic rules, imitation learning and reinforcement learning are trained based on this dataset. Although the authors do not fine-tune LLM-based agents, we believe that the dataset proposed in this paper holds immense potential to enhance the capabilities of agents in the field of web shopping. In EduChat [87], the authors aim to enhance the educational functions of LLMs, such as open-domain question answering, essay assessment, Socratic teaching, and emotional support. They fine-tune LLMs based on human annotated datasets that cover various educational scenarios and tasks.

• *Fine-tuning with LLM Generated Datasets*. Building human-annotated datasets typically requires recruiting people, which can be costly, especially when dealing with large-scale annotation tasks. Considering that LLMs can achieve human-like capabilities in a wide range of tasks, a natural idea is using LLMs to accomplish the annotation task. While the datasets produced from this method can be not as perfect as the human annotated ones, it is much cheaper, and can be leveraged to generate more samples. For example, in ToolBench [14], to enhance the tool-using capability of open-source LLMs, the authors collect 16,464 real-world APIs spanning 49 categories from the RapidAPI Hub. They used these APIs to prompt ChatGPT to generate diverse instructions, covering both single-tool and multi-tool scenarios. Based on the obtained dataset, the authors fine-tune LLaMA [9], and obtain significant performance improvement in terms of tool using. In [83], to empower the agent with social capability, the authors design a sandbox, and deploy multiple agents to interact with each other. Given a social question, the central agent first generates initial responses. Then, it shares the responses to its nearby agents for collecting their feedback. Based on the feedback as well as its detailed explanations, the central agent revise its initial responses to make them more consistent with social norms. In this process, the authors collect a large amount of agent social interaction data, which is then leveraged to fine-tune the LLMs.

• *Fine-tuning with Real-world Datasets*. In addition to building datasets based on human or LLM annotations, directly using real-world datasets to fine-tune the agent is also a common strategy. For example, in MIND2WEB [88], the authors collect a large amount of real-world datasets to enhance the agent capability in the web domain. In contrast to prior studies, the dataset presented in this paper encompasses diverse tasks, real-world scenarios, and comprehensive user interaction patterns. Specifically, the authors collect over 2,000 open-ended tasks from 137 real-world websites spanning 31 domains. Using this dataset, the authors fine-tune LLMs to enhance their performance on web-related tasks such as movie discovery and ticket booking. Similarly, in SQL-PaLM [89], researchers fine-tune PaLM-2 using cross-domain, large-scale text-to-SQL datasets, including Spider and BIRD. The resulting model achieves notable performance improvements on text-to-SQL tasks, demonstrating the effectiveness of real-world datasets for domain-specific applications.

Capability Acquisition without Fine-tuning: In the era of tradition machine learning, the model capability is mainly acquired by learning from datasets, where the knowledge is encoded into the model parameters. In the era of LLMs, the model capability can be acquired either by training/fine-tuning the model parameters or designing delicate prompts

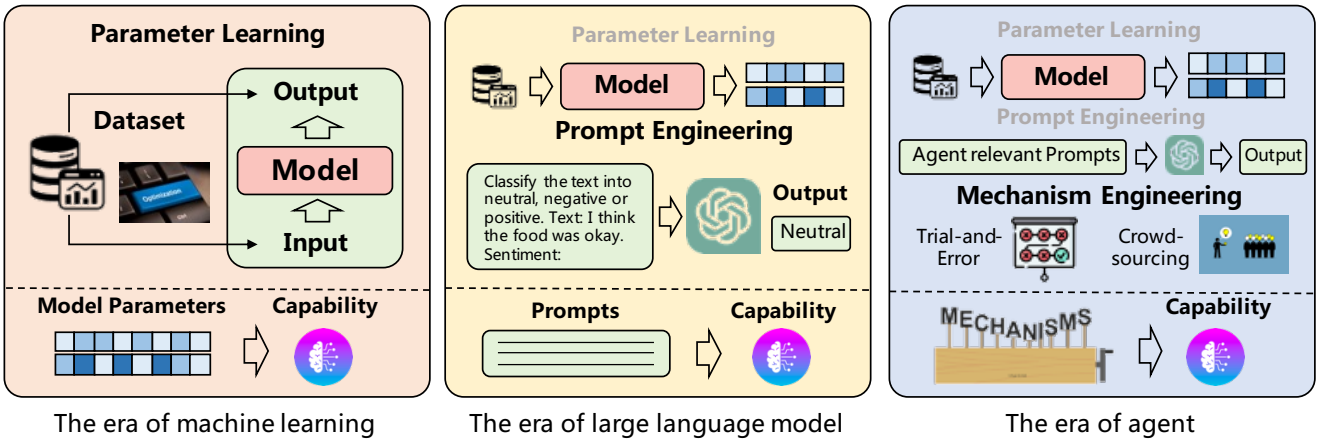

Fig. 4 Illustration of transitions in strategies for acquiring model capabilities.

(*i.e.*, prompt engineering). In prompt engineering, one needs to write valuable information into the prompts to enhance the model capability or unleash existing LLM capabilities. In the era of agents, the model capability can be acquired based on three strategies: (1) model fine-tuning, (2) prompt engineering and (3) designing proper agent evolution mechanisms (we called it as *mechanism engineering*). Mechanism engineering is a broad concept that involves developing specialized modules, introducing novel working rules, and other strategies to enhance agent capabilities. For clearly understanding the transitions of model capability acquisition strategies, we illustrate them in Figure 4. In the following, we detail prompting engineering and mechanism engineering.

• *Prompting Engineering.* Due to the strong language comprehension capabilities, people can directly interact with LLMs using natural languages. This introduces a novel strategy for enhancing agent capabilities, that is, one can describe the desired capability using natural language and then use it as prompts to influence LLM actions. For example, in CoT [45], in order to empower the agent with the capability for complex task reasoning, the authors present the intermediate reasoning steps as few-shot examples in the prompt. Similar techniques are also used in CoT-SC [51] and ToT [52]. In RLP [30], the authors aim to enhance an agent's self-awareness in conversations by prompting LLMs with the agent's beliefs about both its own and the listeners' mental states. This approach results in more engaging and adaptive utterances. Furthermore, the incorporation of the target mental states of listeners allows the agent to formulate more strategic plans. Retroformer [90] presents a retrospective model that enables the agent to generate reflections on its past failures. The reflections are integrated into the prompt of LLMs to guide the agent's future actions. Additionally, this model utilizes reinforcement learning to iteratively improve the retrospective model, thereby refining the LLM prompt.

• *Mechanism Engineering.* Unlike model fine-tuning and prompt engineering, mechanism engineering is a unique strategy to enhance agent capability. In the following, we present several representative methods of mechanism engineering.

(1) *Trial-and-error.* In this method, the agent first performs an action, and subsequently, a pre-defined critic is invoked to judge the action. If the action is deemed unsatisfactory, then the agent reacts by incorporating the critic's feedback. For example, in

RAH [91], the agent serves as a user assistant in recommender systems. One of the agent's crucial roles is to simulate human behavior and generate responses on behalf of the user. To fulfill this objective, the agent first generates a predicted response and then compares it with the real human feedback. If the predicted response and the real human feedback differ, the critic generates failure information, which is subsequently incorporated into the agent's next action. Similarly, in DEPS [33], the agent first designs a plan to accomplish a given task. In the plan execution process, if an action fails, the explainer generates specific details explaining the cause of the failure. This information is then incorporated by the agent to redesign the plan. In RoCo [92], the agent first proposes a sub-task plan and a path of 3D waypoints for each robot in a multi-robot collaboration task. The plan and waypoints are then validated by a set of environment checks, such as collision detection and inverse kinematics. If any of the checks fail, the feedback is appended to each agent's prompt and another round of dialog begins. The agents use LLMs to discuss and improve their plan and waypoints until they pass all validations. PREFER [93] extends this idea by leveraging LLMs to generate detailed feedback when the agent underperforms, enabling iterative refinement and performance improvement.

(2) *Crowd-sourcing.* In [94], the authors design a debating mechanism that leverages the wisdom of crowds to enhance agent capabilities. To begin with, different agents provide separate responses to a given question. If their responses are not consistent, they will be prompted to incorporate the solutions from other agents and provide an updated response. This iterative process continues until reaching a final consensus answer. In this method, the capability of each agent is enhanced by understanding and incorporating the other agents' opinions.

(3) *Experience Accumulation.* In GITM [16], the agent does not know how to solve a task in the beginning. Then, it makes explorations, and once it has successfully accomplished a task, the actions used in this task are stored into the agent memory. In the future, if the agent encounters a similar task, then the relevant memories are extracted to complete the current task. In this process, the improved agent capability comes from the specially designed memory accumulation and utilization mechanisms. Voyager [38] introduces a skill library, where executable codes for specific skills are refined through interactions with the environment, enabling efficient task execution over time. In AppAgent [95], the agent is designed to interact with apps in a manner akin to human users, learning through both autonomous exploration and observation of human demonstrations. Throughout this process, it constructs a knowledge base, which serves as a reference for performing intricate tasks across various applications on a mobile phone. In MemPrompt [96], the users are requested to provide feedback in natural language regarding the problem-solving intentions of the agent, and this feedback is stored in memory. When the agent encounters similar tasks, it attempts to retrieve related memories to generate more suitable responses.

(4) *Self-driven Evolution.* This method allows agents to autonomously improve through self-directed learning and feedback mechanisms. LMA3 [97] enables the agent to autonomously set goals for itself, and gradually improve its capability by exploring the environment and receiving feedback from a reward function. Following this mechanism, the agent can acquire knowledge and develop capabilities according to its own preferences. SALLM-MS [98] integrates advanced LLMs like GPT-4 into a multi-agent system, agents can adapt and perform

complex tasks, showcasing advanced communication capabilities, thereby realizing self-driven evolution in their interactions with the environment. In CLMTWA [99], by using a large language model as a teacher and a weaker language model as a student, the teacher can generate and communicate natural language explanations to improve the student's reasoning skills via theory of mind. The teacher can also personalize its explanations for the student and intervene only when necessary, based on the expected utility of intervention. Meanwhile, NLSOM [100],leverages natural language collaboration between agents, dynamically adjusting roles, tasks, and relationships based on feedback to solve problems beyond the scope of a single agent.

Remark. Upon comparing the aforementioned strategies for agent capability acquisition, we can find that the fine-tuning method improves the agent capability by adjusting model parameters, which can incorporate a large amount of task-specific knowledge, but is only suitable for open-source LLMs. The method without fine-tuning usually enhances the agent capability based on delicate prompting strategies or mechanism engineering. They can be used for both open- and closed-source LLMs. However, due to the limitation of the input context window of LLMs, they cannot incorporate too much task information. In addition, the designing spaces of the prompts and mechanisms are extremely large, which makes it not easy to find optimal solutions.

In the above sections, we have detailed the construction of LLM-based agents, where we focus on two aspects including the architecture design and capability acquisition. We present the correspondence between existing work and the above taxonomy in Table 1. It should be noted that, for the sake of integrity, we have also incorporated several studies, which do not explicitly mention LLM-based agents but are highly related to this area.

3 LLM-based Autonomous Agent Application

Owing to the strong language comprehension, complex task reasoning, and common sense understanding capabilities, LLM-based autonomous agents have shown significant potential to influence multiple domains. This section provides a succinct summary of previous studies, categorizing them according to their applications in three distinct areas: social science, natural science, and engineering (see the left part of Figure 5 for a global overview).

3.1 Social Science

Social science is one of the branches of science, devoted to the study of societies and the relationships among individuals within those societies. LLM-based autonomous agents can promote this domain by leveraging their impressive human-like understanding, thinking and task solving capabilities. In the following, we discuss several key areas that can be affected by LLM-based autonomous agents.

Psychology: For the domain of psychology, LLM-based agents can be leveraged for conducting simulation experiments, providing mental health support and so on [101–104]. For example, in [101], the authors assign LLMs with different profiles, and let them complete psychology experiments. From the results, the authors find that LLMs are capable of generating results that align with those from studies involving human participants. Additionally, it was observed that larger models tend to deliver more accurate simulation results compared to their smaller counterparts. An interesting discovery is that, in many experiments, models like ChatGPT and GPT-4 can provide too perfect estimates (called

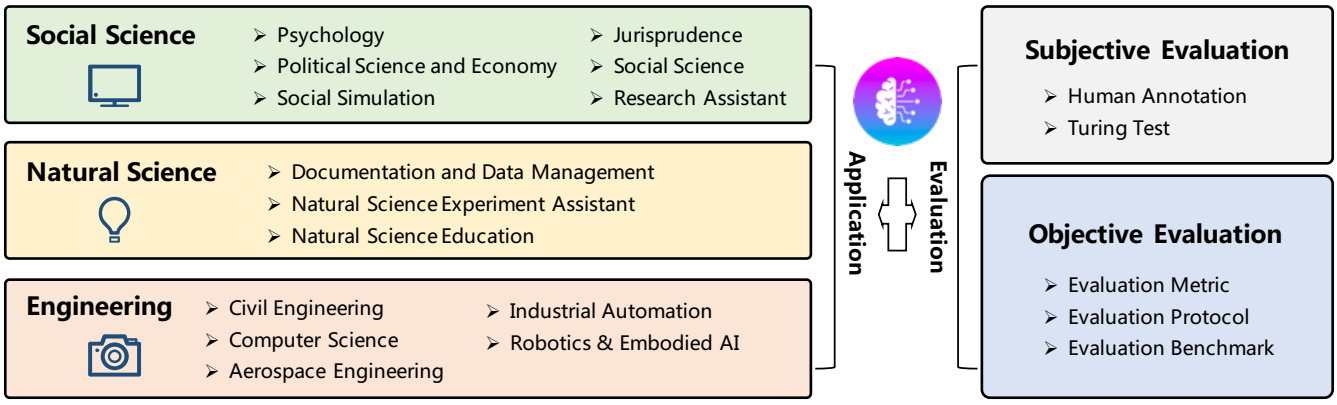

Fig. 5 The applications (left) and evaluation strategies (right) of LLM-based agents.

"hyper-accuracy distortion"), which may influence the downstream applications. In [103], the authors systematically analyze the effectiveness of LLM-based conversation agents for mental well-being support. They collect 120 posts from Reddit, and find that such agents can help users cope with anxieties, social isolation and depression on demand. At the same time, they also find that the agents may produce harmful contents sometimes.

Political Science and Economy: LLM-based agents can also be utilized to study political science and economy [29, 104, 105]. In [29], LLM-based agents are utilized for ideology detection and predicting voting patterns. In [104], the authors focuses on understanding the discourse structure and persuasive elements of political speech through the assistance of LLM-based agents. In [105], LLM-based agents are provided with specific traits such as talents, preferences, and personalities to explore human economic behaviors in simulated scenarios.

Social Simulation: Previously, conducting experiments with human societies is often expensive, unethical, or even infeasible. With the ever prospering of LLMs, many people explore to build virtual environment with LLM-based agents to simulate social phenomena, such as the propagation of harmful information, and so on [20, 34, 78, 80, 106–109]. For example, Social Simulacra [80] simulates an online social community and explores the potential of utilizing agent-based simulations to aid decision-makers to improve community regulations. [106,107] investigates the potential impacts of different behavioral characteristics of LLM-based agents in social networks. Generative Agents [20] and AgentSims [34] construct multiple agents in a virtual town to simulate the human daily life. SocialAI School [108] employs LLM-based agents to simulate and investigate the fundamental social cognitive skills during the course of child development. S^3 [78] builds a social network simulator, focusing on the propagation of information, emotion and attitude. CGMI [110] is a framework for multi-agent simulation. CGMI maintains the personality of the agents through a tree structure and constructs a cognitive model. The authors simulated a classroom scenario using CGMI.

Jurisprudence: LLM-based agents can serve as aids in legal decision-making processes, facilitating more informed judgements [111, 112]. Blind Judgement [112] employs several language models to simulate the decision-making processes of multiple judges. It gathers diverse opinions and consolidates the outcomes through a voting mechanism. ChatLaw [111] is a prominent Chinese le-

gal model based on LLM. It adeptly supports both database and keyword search strategies, specifically designed to mitigate the hallucination issue prevalent in such models. In addition, this model also employs self-attention mechanism to enhance the LLM's capability via mitigating the impact of reference inaccuracies.

Research Assistant: Beyond their application in specialized domains, LLM-based agents are increasingly adopted as versatile assistants in the broad field of social science research [104, 113]. In [104], LLM-based agents offer multifaceted assistance, ranging from generating concise article abstracts and extracting pivotal keywords to crafting detailed scripts for studies, showcasing their ability to enrich and streamline the research process. Meanwhile, in [113], LLM-based agents serve as a writing assistant, demonstrating their capability to identify novel research inquiries for social scientists, thereby opening new avenues for exploration and innovation in the field. These examples highlight the potential of LLM-based agents in enhancing the efficiency, creativity, and breadth of social science research.

3.2 Natural Science

Natural science is one of the branches of science concerned with the description, understanding and prediction of natural phenomena, based on empirical evidence from observation and experimentation. With the ever prospering of LLMs, the application of LLM-based agents in natural sciences becomes more and more popular. In the following, we present many representative areas, where LLM-based agents can play important roles.

Documentation and Data Management: Natural scientific research often involves the collection, organization, and synthesis of substantial amounts of literature, which requires a significant dedication of time and human resources. LLM-based agents have shown strong capabilities on language understanding and employing tools such as the internet and databases for text processing. These capabilities empower the agent to excel in tasks related to documentation and data management. In [114], the agent can efficiently query and utilize internet information to complete tasks such as question answering and experiment planning. ChatMOF [115] utilizes LLMs to extract important information from text descriptions written by humans. It then formulates a plan to apply relevant tools for predicting the properties and structures of metal-organic frameworks. ChemCrow [76] utilizes chemistry-related databases to both validate the precision of compound representations and identify potentially dangerous substances. This functionality enhances the reliability and comprehensiveness of scientific inquiries by ensuring the accuracy of the data involved.

Experiment Assistant: LLM-based agents have the ability to independently conduct experiments, making them valuable tools for supporting scientists in their research projects [76,114]. For instance, [114] introduces an innovative agent system that utilizes LLMs for automating the design, planning, and execution of scientific experiments. This system, when provided with the experimental objectives as input, accesses the Internet and retrieves relevant documents to gather the necessary information. It subsequently utilizes Python code to conduct essential calculations and carry out the following experiments. ChemCrow [76] incorporates 17 carefully developed tools that are specifically designed to assist researchers in their chemical research. Once the input objectives are received, ChemCrow provides valuable recommendations for experimental procedures, while also emphasizing any potential safety

risks associated with the proposed experiments.

Natural Science Education: LLM-based agents can communicate with humans fluently, often being utilized to develop agent-based educational tools. For example, [114] develops agent-based education systems to facilitate students learning of experimental design, methodologies, and analysis. The objective of these systems is to enhance students' critical thinking and problem-solving skills, while also fostering a deeper comprehension of scientific principles. Math Agents [116] can assist researchers in exploring, discovering, solving and proving mathematical problems. Additionally, it can communicate with humans and aids them in understanding and using mathematics. [117] utilize the capabilities of CodeX [118] to automatically solve and explain university-level mathematical problems, which can be used as education tools to teach students and researchers. CodeHelp [119] is an education agent for programming. It offers many useful features, such as setting course-specific keywords, monitoring student queries, and providing feedback to the system. EduChat [87] is an LLM-based agent designed specifically for the education domain. It provides personalized, equitable, and empathetic educational support to teachers, students, and parents through dialogue. FreeText [120] is an agent that utilizes LLMs to automatically assess students' responses to open-ended questions and offer feedback.

3.3 Engineering

LLM-based autonomous agents have shown great potential in assisting and enhancing engineering research and applications. In this section, we review and summarize the applications of LLM-based agents in several major engineering domains.

Computer Science & Software Engineering: In the field of computer science and software engineering, LLM-based agents offer potential for automating coding, testing, debugging, and documentation generation [18, 23, 24, 126–128]. ChatDev [18] proposes an end-to-end framework, where multiple agent roles communicate and collaborate through natural language conversations to complete the software development life cycle. This framework demonstrates efficient and cost-effective generation of executable software systems. MetaGPT [23] abstracts multiple roles, such as product managers, architects, project managers, and engineers, to supervise code generation process and enhance the quality of the final output code. This enables low-cost software development. [24] presents a self-collaboration framework for code generation using LLMs. In this framework, multiple LLMs are assumed to be distinct "experts" for specific subtasks. They collaborate and interact according to specified instructions, forming a virtual team that facilitates each other's work. Ultimately, the virtual team collaboratively addresses code generation tasks without requiring human intervention. LLIFT [141] employs LLMs to assist in conducting static analysis, specifically for identifying potential code vulnerabilities. This approach effectively manages the trade-off between accuracy and scalability. ChatEDA [123] is an agent developed for electronic design automation (EDA) to streamline the design process by integrating task planning, script generation, and execution. CodeHelp [119] is an agent designed to assist students and developers in debugging and testing their code. Its features include providing detailed explanations of error messages, suggesting potential fixes, and ensuring the accuracy of the code. Pentest [125] is a penetration testing tool based on LLMs, which can effectively identify common vulnerabilities, and interpret source code to develop exploits. D-Bot [122] utilizes the capa-

Table 2 Representative applications of LLM-based autonomous agents.

	Domain	Work
Social Science	Psychology	TE [101], Akata et al. [102], Ziems et al. [104], Ma et al. [103]
	Political Science and Economy	Argyle et al. [29], Horton [105], Ziems et al. [104]
	Social Simulation	Social Simulacra [80], Generative Agents [20], SocialAI School [108], AgentSims [34], S^3 [78], Williams et al. [109], Li et al. [106], Chao et al. [107]
	Jurisprudence	ChatLaw [111], Blind Judgement [112]
	Research Assistant	Ziems et al. [104], Bail et al. [113]
Natural Science	Documentation and Data Management	ChemCrow [76], ChatMOF [115], Boiko et al. [114]
	Experiment Assistant	ChemCrow [76], Boiko et al. [114], Grossmann et al. [121]
	Natural Science Education	ChemCrow [76], CodeHelp [119], Boiko et al. [114], MathAgent [116], Drori et al. [117], EduChat [87], FreeText [120]
Engineering	CS & SE	RestGPT [71], Self-collaboration [24], SQL-PALM [89], RAH [91], D-Bot [122], RecMind [53], ChatEDA [123], InteRecAgent [124], PentestGPT [125], CodeHelp [119], SmolModels [126], DemoGPT [127], GPTEngineer [128]
	Industrial Automation	GPT4IA [129], IELLM [130]
	Robotics & Embodied AI	ProAgent [131], LLM4RL [132], PET [133], REMEMBERER [134], DEPS [33], Unified Agent [135], SayCan [79], TidyBot [136], RoCo [92], SayPlan [31], TaPA [137], Dasgupta et al. [138], DECKARD [139], Dialogue shaping [140]

bilities of LLMs to systematically assess potential root causes of anomalies in databases. Through the implementation of a tree of thought approach, D-Bot enables LLMs to backtrack to previous steps in case the current step proves unsuccessful, thus enhancing the accuracy of the diagnosis process.

Industrial Automation: In the field of industrial automation, LLM-based agents can be used to achieve intelligent planning and control of production processes. [129] proposes a novel framework that integrates LLMs with digital twin systems to accommodate flexible production needs. The framework leverages prompt engineering techniques to create LLM agents that can adapt to specific tasks based on the information provided by digital twins. These agents can coordinate a series of atomic functionalities and skills to complete production tasks at different levels. This research demonstrates the potential of integrating LLMs into industrial automation systems, providing innovative solutions for more agile, flexible and adaptive production processes. IELLM [130] showcases a case study on LLMs' role in the oil and gas industry, covering applications like factory automation and PLC programming.

Robotics & Embodied Artificial Intelligence: Recent works have advanced the development of more efficient reinforcement learning agents for

robotics and embodied artificial intelligence [16, 38, 79, 132–135, 137–140]. These efforts focus on enhancing autonomous agents' capabilities in planning, reasoning, and collaboration within embodied environments. For instance, [138] proposes the Planner-Actor-Reporter paradigm for embodied reasoning and task planning. DECKARD [139] introduces the Planner-Actor-Reporter paradigm, which facilitates embodied reasoning and task planning by decoupling the agent's planning, execution, and reporting processes. TaPA [137] constructs a multimodal dataset comprising multi-view RGB images of indoor scenes, human instructions, and corresponding plans to fine-tune LLMs. The fine-tuned models align visual perception with task planning, enabling them to generate more executable plans and significantly improving their performance in visually grounded tasks.

To overcome the physical constraints, the agents can generate executable plans and accomplish long-term tasks by leveraging multiple skills. In terms of control policies, SayCan [79] focuses on investigating a wide range of manipulation and navigation skills utilizing a mobile manipulator robot. Taking inspiration from typical tasks encountered in a kitchen environment, it presents a comprehensive set of 551 skills that cover seven skill families and 17 objects. These skills encompass various actions such as picking, placing, grasping, and manipulating objects, among others. TidyBot [136] is an embodied agent designed to personalize household cleanup tasks. It can learn users' preferences on object placement and manipulation methods through textual examples.

To promote the application of LLM-based autonomous agents, researchers have also introduced many open-source libraries, based on which the developers can quickly implement and evaluate agents according to their customized requirements [19, 82, 127, 142–155]. For example, LangChain [147] is an open-source framework that automates coding, testing, debugging, and documentation generation tasks. By integrating language models with data sources and facilitating interaction with the environment, LangChain enables efficient and cost-effective software development through natural language communication and collaboration among multiple agent roles. Based on LangChain, XLang [145] provides a comprehensive set of tools and a fully integrated user interface. It focuses on executable language grounding, enabling the conversion of natural language instructions into code or action sequences that interact seamlessly with various environments, including databases, web applications, and physical robots. AutoGPT [82] is an agent that is fully automated. It sets one or more goals, breaks them down into corresponding tasks, and cycles through the tasks until the goal is achieved. WorkGPT [148] is an agent framework similar to AutoGPT and LangChain. By providing it with an instruction and a set of APIs, it engages in back-and-forth conversations with AI until the instruction is completed. GPT-Engineer [128] and DemoGPT [127] are open-source projects that focus on automating code generation through prompts to complete development tasks. SmolModels [126] offers a family of compact language models suitable for various tasks. AGiXT [144] is a dynamic AI automation platform that efficiently manages instructions and executes complex tasks across various AI providers, integrating adaptive memory, smart features, and a versatile plugin system. AgentVerse [156] is a versatile framework that facilitates researchers in creating customized LLM-based agent simulations efficiently. GPT Researcher [150] is an experimental application that leverages LLMs to efficiently

develop research questions, trigger web crawls to gather information, summarize sources, and aggregate summaries. BMTools [151] provides a platform for community-driven tool building and sharing. It supports various types of tools, enables simultaneous task execution using multiple tools, and offers a simple interface for loading plugins via URLs, fostering easy development and contribution to the BMTools ecosystem.

Remark. Utilization of LLM-based agents in supporting above applications may also entail risks and challenges. On one hand, LLMs themselves may be susceptible to illusions and other issues, occasionally providing erroneous answers, leading to incorrect conclusions, experimental failures, or even posing risks to human safety in hazardous experiments. Therefore, during experimentation, users must possess the necessary expertise and knowledge to exercise appropriate caution. On the other hand, LLM-based agents could potentially be exploited for malicious purposes, such as the development of chemical weapons, necessitating the implementation of security measures, such as human alignment, to ensure responsible and ethical use.

In summary, in the above sections, we introduce the typical applications of LLM-based autonomous agents in three important domains. To facilitate a clearer understanding, we have summarized the relationship between previous studies and their respective applications in Table 2.

4 LLM-based Autonomous Agent Evaluation

Similar to LLMs themselves, evaluating the effectiveness of LLM-based autonomous agents is a challenging task. This section outlines two prevalent approaches to evaluation: subjective and objective methods. For a comprehensive overview, please refer to the right portion of Figure 5.

4.1 Subjective Evaluation

Subjective evaluation measures the agent capabilities based on human judgements [20,22,29,80,157]. It is suitable for the scenarios where there are no evaluation datasets or it is very hard to design quantitative metrics, for example, evaluating the agent's intelligence or user-friendliness. In the following, we present two commonly used strategies for subjective evaluation.

Human Annotation: This evaluation method involves human evaluators directly scoring or ranking the outputs generated by various agents [22,29,104]. For example, in [20], the authors engage numerous annotators by asking 25 questions that explore their abilities across five key areas directly related to agent capabilities. In [80], annotators are asked to determine whether the specifically designed models can significantly enhance the development of rules within online communities.

Turing Test: This evaluation strategy necessitates that human evaluators differentiate between outputs produced by agents and those created by humans. If, in a given task, the evaluators cannot separate the agent and human results, it demonstrates that the agent can achieve human-like performance on this task. For instance, researchers in [29] conduct experiments on free-form Partisan text, and the human evaluators are asked to guess whether the responses are from human or LLM-based agent.

Remark. LLM-based agents are usually designed to serve humans. Thus, subjective agent evaluation plays a critical role, since it reflects human criterion. However, this strategy also faces issues such as high costs, inefficiency, and population bias. To address these issues, a growing number of researchers are in-

vestigating the use of LLMs themselves as intermediaries for carrying out these subjective assessments. For example, in ChemCrow [76], researchers assess the experimental results using GPT. They consider both the completion of tasks and the accuracy of the underlying processes. Similarly, ChatEval [158] introduces a novel approach by employing multiple agents to critique and assess the results generated by various candidate models in a structured debate format. This innovative use of LLMs for evaluation purposes holds promise for enhancing both the credibility and applicability of subjective assessments in the future. As LLM technology continues to evolve, it is anticipated that these methods will become increasingly reliable and find broader applications, thereby overcoming the current limitations of direct human evaluation.

4.2 Objective Evaluation

Objective evaluation refers to assessing the capabilities of LLM-based autonomous agents using quantitative metrics that can be computed, compared and tracked over time. In contrast to subjective evaluation, objective metrics aim to provide concrete, measurable insights into the agent performance. For conducting objective evaluation, there are three important aspects, that is, the evaluation metrics, protocols and benchmarks. In the following, we introduce these aspects more in detail.

Metrics: In order to objectively evaluate the effectiveness of the agents, designing proper metrics is significant, which may influence the evaluation accuracy and comprehensiveness. Ideal evaluation metrics should precisely reflect the quality of the agents, and align with the human feelings when using them in real-world scenarios. In existing work, we can conclude the following representative evaluation metrics. (1) *Task success metrics:* These metrics measure how well an agent can complete tasks and achieve goals. Common metrics include success rate [12, 22, 58, 60], reward/score [22, 60, 161], coverage [16], and accuracy/error rate [18, 40, 80, 101]. Depending on the scenario, accuracy may reflect aspects such as program executability [18] or task validity [101]. Higher values across these task success metrics indicate greater task completion ability. (2) *Human similarity metrics:* These metrics quantify the degree to which the agent behaviors closely resembles those of humans by emphasizing various aspects related to human traits, such as coherent [104], fluent [104], dialogue similarities with human [80] and human acceptance rate [101]. Higher similarity suggests better human simulation performance. (3) *Efficiency metrics:* In contrast to the aforementioned metrics used to evaluate the agent effectiveness, these metrics aim to assess the efficiency of agent. Commonly considered metrics encompass the cost associated with development [18] and training efficiency [16, 38].

Protocols: In addition to the evaluation metrics, another important aspect for objective evaluation is how to leverage these metrics. In the previous work, we can identify the following commonly used evaluation protocols: (1) *Real-world simulation:* In this method, the agents are evaluated within immersive environments like games and interactive simulators. The agents are required to perform tasks autonomously, and then metrics like task success rate and human similarity are leveraged to evaluate the capability of the agents based on their trajectories and completed objectives [12, 16, 22, 33, 38, 60, 86, 161, 164, 168]. By simulating real-world scenarios, this approach aims to provide a comprehensive evaluation of the agents' practical capabilities. (2) *Social evaluation*: This method utilizes

Table 3 For subjective evaluation, we use ① and ② to represent human annotation and the Turing test, respectively. For objective evaluation, we use ①, ②, ③, and ④ to represent real-world simulation, social evaluation, multi-task evaluation, and software testing, respectively. "✓" indicates that the evaluations are based on benchmarks.

Model	Subjective	Objective	Benchmark	Time
WebShop [86]	-	① ③	✓	07/2022
Social Simulacra [80]	①	②	-	08/2022
TE [101]	-	②	-	08/2022
LIBRO [159]	-	④	-	09/2022
ReAct [60]	-	①	✓	10/2022
Argyle et al. [29]	②	② ③	-	02/2023
DEPS [33]	-	①	✓	02/2023
Jalil et al. [160]	-	④	-	02/2023
Reflexion [12]	-	① ③	-	03/2023
IGLU [161]	-	①	✓	04/2023
Generative Agents [20]	①	-	-	04/2023
ToolBench [151]	-	③	✓	04/2023
GITM [16]	-	①	✓	05/2023
Two-Failures [162]	-	③	-	05/2023
Voyager [38]	-	①	✓	05/2023
SocKET [163]	-	② ③	✓	05/2023
MobileEnv [164]	-	① ③	✓	05/2023
Clembench [165]	-	① ③	✓	05/2023
Dialop [166]	-	③	✓	06/2023
Feldt et al. [167]	-	④	-	06/2023
CO-LLM [22]	①	①	-	07/2023
Tachikuma [168]	①	① ③	✓	07/2023
RocoBench [92]	-	① ③	✓	07/2023
AgentSims [34]	-	②	-	08/2023
AgentBench [169]	-	③	✓	08/2023
BOLAA [170]	-	③	✓	08/2023
Gentopia [171]	-	③	✓	08/2023
EmotionBench [172]	①	-	✓	08/2023
PTB [125]	-	④	-	08/2023

metrics to assess social intelligence based on the agent interactions in simulated societies. Various approaches have been adopted, such as collaborative tasks to evaluate teamwork skills, debates to analyze argumentative reasoning, and human studies to measure social aptitude [34, 80, 101, 163]. These approaches analyze qualities such as coherence, theory of mind, and social IQ to assess agents' capabilities in areas including cooperation, communication, empathy, and mimicking human social behavior. By subjecting agents to complex interactive settings, social evaluation provides valuable insights into agents' higher-level social cognition.

(3) *Multi-task evaluation:* In this method, people use a set of diverse tasks from different domains to evaluate the agent, which can effectively measure the agent generalization capability in open-domain environments [12,29,86,151,162–164,169,170]. (4) *Software testing:* In this method, researchers evaluate the agents by letting them conduct tasks such as software testing tasks, such as generating test cases, reproducing bugs, debugging code, and interacting with developers and external tools [159, 160, 167]. Then, one can use metrics like test coverage and bug detection rate to measure the effectiveness of LLM-based agents.

Benchmarks: Given the metrics and protocols, a crucial aspect of evaluation is the selection of appropriate benchmarks. Over time, various benchmarks have been introduced to assess the capabilities of LLM-based agents across diverse domains and scenarios. Many studies employ environments such as ALFWorld [60], IGLU [161], and Minecraft [16, 33, 38] to evaluate agent capabilities in interactive and task-oriented simulations. Tachikuma [168] evaluates LLMs' ability to infer and understand complex interactions involving multiple characters and novel objects through TRPG game logs. AgentBench [169] provides a comprehensive framework for evaluating LLMs as autonomous agents across diverse environments. It represents the first systematic assessment of LLMs as agents on real-world challenges across diverse domains. SocKET [163] is a comprehensive benchmark for evaluating the social capabilities of LLMs across 58 tasks covering five categories of social information such as humor and sarcasm, emotions and feelings, credibility, etc. AgentSims [34] is a versatile framework for evaluating LLM-based agents, where one can flexibly design the agent planning, memory and action strategies, and measure the effectiveness of different agent modules in interactive environments. ToolBench [151] focuses on assessing and enhancing language models' ability to use tools, featuring 16,464 real-world RESTful APIs and diverse instructions tailored for single- and multi-tool scenarios. WebShop [86] develops a benchmark for evaluating LLM-based agents in terms of their capabilities for product search and retrieval, which is constructed using a collection of 1.18 million real-world items. Mobile-Env [164] serves as an extendable interactive platform designed to evaluate the multi-step interaction capabilities of LLM-based agents. WebArena [173] offers a comprehensive website environment that spans multiple domains. Its purpose is to evaluate agents in an end-to-end fashion and determine the accuracy of their completed tasks. GentBench [171] is crafted to evaluate the agent capabilities, including their reasoning, safety, and efficiency, when utilizing tools to complete complex tasks. RocoBench [92] comprises six tasks that evaluate multi-agent collaboration across diverse scenarios, emphasizing communication and coordination strategies to assess adaptability and generalization in cooperative robotics. EmotionBench [172] evaluates the emotion appraisal ability of LLMs, i.e., how their feelings change when presented with specific situations. It collects over 400 situations that elicit eight negative emotions and measures the emotional states of LLMs and human subjects using self-report scales. PEB [125] is tailored for assessing LLM-based agents in penetration testing scenarios, comprising 13 diverse targets from leading platforms. It offers a structured evaluation across varying difficulty levels, reflecting real-world challenges for agents. ClemBench [165] contains five Dialogue Games to assess LLMs' ability as a player. E2E [174] serves as an end-to-end benchmark for testing the accuracy and usefulness of chatbots.

Remark. Objective evaluation facilitates the quantitative analysis of capabilities in LLM-based agents through a variety of metrics. While current techniques can not perfectly measure all types of agent capabilities, objective evaluation provides essential insights that complement subjective assessment. Continued advancements in benchmarks and methodologies for objective evaluation will enhance the development and understanding of LLM-based autonomous agents further.

In the above sections, we introduce both subjective and objective strategies for LLM-based au-

tonomous agent evaluation. The evaluation of the agents play significant roles in this domain. However, both subjective and objective evaluation have their own strengths and weakness. Maybe, in practice, they should be combined to comprehensively evaluate the agents. We summarize the correspondence between the previous work and these evaluation strategies in Table 3.

5 Related Surveys

With the vigorous development of large language models, a variety of comprehensive surveys have emerged, providing detailed insights into various aspects. [175] extensively introduces the background, main findings, and mainstream technologies of LLMs, encompassing a vast array of existing works. On the other hand, [176] primarily focus on the applications of LLMs in various downstream tasks and the challenges associated with their deployment. Aligning LLMs with human intelligence is an active area of research to address concerns such as biases and illusions. [177] have compiled existing techniques for human alignment, including data collection and model training methodologies. Reasoning is a crucial aspect of intelligence, influencing decision-making, problem-solving, and other cognitive abilities. [178] presents the current state of research on LLMs' reasoning abilities, exploring approaches to improve and evaluate their reasoning skills. [179] propose that language models can be enhanced with reasoning capabilities and the ability to utilize tools, termed Augmented Language Models (ALMs). They conduct a comprehensive review of the latest advancements in ALMs. As the utilization of large-scale models becomes more prevalent, evaluating their performance is increasingly critical. [180] shed light on evaluating LLMs, addressing what to evaluate, where to evaluate, and how to assess their performance in downstream tasks and societal impact. [181] also discusses the capabilities and limitations of LLMs in various downstream tasks. The aforementioned research encompasses various aspects of large models, including training, application, and evaluation. However, prior to this paper, no work has specifically focused on the rapidly emerging and highly promising field of LLM-based Agents. In this study, we have compiled 100 relevant works on LLM-based Agents, covering their construction, applications, and evaluation processes.

6 Challenges

While previous work on LLM-based autonomous agent has obtained many remarkable successes, this field is still at its initial stage, and there are several significant challenges that need to be addressed in its development. In the following, we present many representative challenges.

6.1 Role-playing Capability

Different from traditional LLMs, autonomous agent usually has to play as specific roles (*e.g.*, program coder, researcher and chemist) for accomplishing different tasks. Thus, the capability of the agent for role-playing is very important. Although LLMs can effectively simulate many common roles such as movie reviewers, there are still various roles and aspects that they struggle to capture accurately. To begin with, LLMs are usually trained based on web-corpus, thus for the roles which are seldom discussed on the web or the newly emerging roles, LLMs may not simulate them well. In addition, previous research [30] has shown that existing LLMs may not well model the human cognitive psychology characters, leading to the lack of self-awareness

in conversation scenarios. Potential solution to these problems may include fine-tuning LLMs or carefully designing the agent prompts/architectures [182]. For example, one can firstly collect real-human data for uncommon roles or psychology characters, and then leverage it to fine-tune LLMs. However, how to ensure that fine-tuned model still perform well for the common roles may pose further challenges. Beyond fine-tuning, one can also design tailored agent prompts/architectures to enhance the capability of LLM on role-playing. However, finding the optimal prompts/architectures is not easy, since their designing spaces are too large.

6.2 Generalized Human Alignment

Human alignment has been discussed a lot for traditional LLMs. In the field of LLM-based autonomous agent, especially when the agents are leveraged for simulation, we believe this concept should be discussed more in depth. In order to better serve human-beings, traditional LLMs are usually fine-tuned to be aligned with correct human values, for example, the agent should not plan to make a bomb for avenging society. However, when the agents are leveraged for real-world simulation, an ideal simulator should be able to honestly depict diverse human traits, including the ones with incorrect values. Actually, simulating the human negative aspects can be even more important, since an important goal of simulation is to discover and solve problems, and without negative aspects means no problem to be solved. For example, to simulate the real-world society, we may have to allow the agent to plan for making a bomb, and observe how it will act to implement the plan as well as the influence of its behaviors. Based on these observations, people can make better actions to stop similar behaviors in real-world society. Inspired by the above case, maybe an important problem for agent-based simulation is how to conduct generalized human alignment, that is, for different purposes and applications, the agent should be able to align with diverse human values. However, existing powerful LLMs including ChatGPT and GPT-4 are mostly aligned with unified human values. Thus, an interesting direction is how to "realign" these models by designing proper prompting strategies.

6.3 Prompt Robustness

To ensure rational behavior in agents, it's a common practice for designers to embed supplementary modules, such as memory and planning modules, into LLMs. However, the inclusion of these modules necessitates the development of more complex prompts in order to facilitate consistent operation and effective communication. Previous research [183, 184] has highlighted the lack of robustness in prompts for LLMs, as even minor alterations can yield substantially different outcomes. This issue becomes more pronounced when constructing autonomous agents, as they encompass not a single prompt but a prompt framework that considers all modules, wherein the prompt for one module has the potential to influence others. Moreover, the prompt frameworks can vary significantly across different LLMs. The development of a unified and resilient prompt framework applicable across diverse LLMs remains a critical and unresolved challenge. There are two potential solutions to the aforementioned problems: (1) manually crafting the essential prompt elements through trial and error, or (2) automatically generating prompts using GPT.

6.4 Hallucination

Hallucination poses a fundamental challenge for LLMs, characterized by the models' tendency to

produce false information with a high level of confidence. This challenge is not limited to LLMs alone but is also a significant concern in the domain of autonomous agents. For instance, in [185], it was observed that when confronted with simplistic instructions during code generation tasks, the agent may exhibit hallucinatory behavior. Hallucination can lead to serious consequences such as incorrect or misleading code, security risks, and ethical issues [185]. To mitigate this issue, incorporating human correction feedback directly into the iterative process of human-agent interaction presents a viable approach [23]. More discussions on the hallucination problem can be seen in [175].

6.5 Knowledge Boundary

A pivotal application of LLM-based autonomous agents lies in simulating diverse real-world human behaviors [20]. The study of human simulation has a long history, and the recent surge in interest can be attributed to the remarkable advancements made by LLMs, which have demonstrated significant capabilities in simulating human behavior. However, it is important to recognize that the power of LLMs may not always be advantageous. Specifically, an ideal simulation should accurately replicate human knowledge. In this context, LLMs may display overwhelming capabilities, being trained on a vast corpus of web knowledge that far exceeds what an average individual might know. The immense capabilities of LLMs can significantly impact the effectiveness of simulations. For instance, when attempting to simulate user selection behaviors for various movies, it is crucial to ensure that LLMs assume a position of having no prior knowledge of these movies. However, there is a possibility that LLMs have already acquired information about these movies. Without implementing appropriate strategies, LLMs may make decisions based on their extensive knowledge, even though real-world users would not have access to the contents of these movies beforehand. Based on the above example, we may conclude that for building believable agent simulation environment, an important problem is how to constrain the utilization of user-unknown knowledge of LLM.

6.6 Efficiency

Due to their autoregressive architecture, LLMs typically have slow inference speeds. However, the agent may need to query LLMs for each action multiple times, such as extracting information from memory, make plans before taking actions and so on. Consequently, the efficiency of agent actions is greatly affected by the speed of LLM inference.

7 Conclusion

In this survey, we systematically summarize existing research in the field of LLM-based autonomous agents. We present and review these studies from three aspects including the construction, application, and evaluation of the agents. For each of these aspects, we provide a detailed taxonomy to draw connections among the existing research, summarizing the major techniques and their development histories. In addition to reviewing the previous work, we also propose several challenges in this field, which are expected to guide potential future directions.

Acknowledgement

This work is supported in part by National Natural Science Foundation of China (No. 62102420), Beijing Outstanding Young Scientist Program NO. BJJWZYJH012019100020098, Intelligent Social

Governance Platform, Major Innovation & Planning Interdisciplinary Platform for the "Double-First Class" Initiative, Renmin University of China, Public Computing Cloud, Renmin University of China, fund for building world-class universities (disciplines) of Renmin University of China, Intelligent Social Governance Platform.

References

1. Mnih V, Kavukcuoglu K, Silver D, Rusu A A, Veness J, Bellemare M G, Graves A, Riedmiller M, Fidjeland A K, Ostrovski G, others . Human-level control through deep reinforcement learning. nature, 2015, 518(7540): 529–533
2. Lillicrap T P, Hunt J J, Pritzel A, Heess N, Erez T, Tassa Y, Silver D, Wierstra D. Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971, 2015
3. Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017
4. Haarnoja T, Zhou A, Abbeel P, Levine S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: International conference on machine learning. 2018, 1861–1870
5. Brown T, Mann B, Ryder N, Subbiah M, Kaplan J D, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A, others . Language models are few-shot learners. Advances in neural information processing systems, 2020, 33: 1877–1901
6. Radford A, Wu J, Child R, Luan D, Amodei D, Sutskever I, others . Language models are unsupervised multitask learners. OpenAI blog, 2019, 1(8): 9
7. Achiam J, Adler S, Agarwal S, Ahmad L, Akkaya I, Aleman F L, Almeida D, Altenschmidt J, Altman S, Anadkat S, others . Gpt-4 technical report. arXiv preprint arXiv:2303.08774, 2023
8. Anthropic . Model card and evaluations for claude models. https://www-files.anthropic.com/production/images/Model-Card-Claude-2.pdf?ref=maginative.com, 2023
9. Touvron H, Lavril T, Izacard G, Martinet X, Lachaux M A, Lacroix T, Rozière B, Goyal N, Hambro E, Azhar F, others . Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971, 2023
10. Touvron H, Martin L, Stone K, Albert P, Almahairi A, Babaei Y, Bashlykov N, Batra S, Bhargava P, Bhosale S, others . Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288, 2023
11. Chen X, Li S, Li H, Jiang S, Qi Y, Song L. Generative adversarial user model for reinforcement learning based recommendation system. In: International Conference on Machine Learning. 2019, 1052–1061
12. Shinn N, Cassano F, Gopinath A, Narasimhan K, Yao S. Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems, 2024, 36
13. Shen Y, Song K, Tan X, Li D, Lu W, Zhuang Y. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. Advances in Neural Information Processing Systems, 2024, 36
14. Qin Y, Liang S, Ye Y, Zhu K, Yan L, Lu Y, Lin Y, Cong X, Tang X, Qian B, others . Toolllm: Facilitating large language models to master 16000+ real-world apis. arXiv preprint arXiv:2307.16789, 2023
15. Schick T, Dwivedi-Yu J, Dessì R, Raileanu R, Lomeli M, Hambro E, Zettlemoyer L, Cancedda N, Scialom T. Toolformer: Language models can teach themselves to use tools. Advances in Neural Information Processing Systems, 2024, 36
16. Zhu X, Chen Y, Tian H, Tao C, Su W, Yang C, Huang G, Li B, Lu L, Wang X, others . Ghost in the minecraft: Generally capable agents for open-world enviroments via large language models with text-based knowledge and memory. arXiv preprint arXiv:2305.17144, 2023
17. Sclar M, Kumar S, West P, Suhr A, Choi Y, Tsvetkov Y. Minding language models'(lack of) theory of mind: A plug-and-play multi-character belief tracker. arXiv preprint arXiv:2306.00924, 2023
18. Qian C, Cong X, Yang C, Chen W, Su Y, Xu J, Liu Z, Sun M. Communicative agents for software development. arXiv preprint arXiv:2307.07924, 2023
19. al. e C. Agentverse. https://github.com/OpenBMB/AgentVerse, 2023
20. Park J S, O'Brien J, Cai C J, Morris M R, Liang P, Bernstein M S. Generative agents: Interactive simulacra of human behavior. In: Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology. 2023, 1–22
21. Wang L, Zhang J, Chen X, Lin Y, Song R, Zhao W X, Wen J R. Recagent: A novel simulation paradigm for recommender systems. arXiv preprint arXiv:2306.02552, 2023
22. Zhang H, Du W, Shan J, Zhou Q, Du Y, Tenenbaum J B, Shu T, Gan C. Building cooperative embodied agents modularly with large language models. arXiv

preprint arXiv:2307.02485, 2023
23. Hong S, Zheng X, Chen J, Cheng Y, Wang J, Zhang C, Wang Z, Yau S K S, Lin Z, Zhou L, others . Metagpt: Meta programming for multi-agent collaborative framework. arXiv preprint arXiv:2308.00352, 2023
24. Dong Y, Jiang X, Jin Z, Li G. Self-collaboration code generation via chatgpt. arXiv preprint arXiv:2304.07590, 2023
25. Safdari M, Serapio-García G, Crepy C, Fitz S, Romero P, Sun L, Abdulhai M, Faust A, Matarić M. Personality traits in large language models. arXiv preprint arXiv:2307.00184, 2023
26. Johnson J A. Measuring thirty facets of the five factor model with a 120-item public domain inventory: Development of the ipip-neo-120. Journal of research in personality, 2014, 51: 78–89
27. John O P, Donahue E M, Kentle R L. Big five inventory. Journal of Personality and Social Psychology, 1991
28. Deshpande A, Murahari V, Rajpurohit T, Kalyan A, Narasimhan K. Toxicity in chatgpt: Analyzing persona-assigned language models. arXiv preprint arXiv:2304.05335, 2023
29. Argyle L P, Busby E C, Fulda N, Gubler J R, Rytting C, Wingate D. Out of one, many: Using language models to simulate human samples. Political Analysis, 2023, 31(3): 337–351
30. Fischer K A. Reflective linguistic programming (rlp): A stepping stone in socially-aware agi (socialagi). arXiv preprint arXiv:2305.12647, 2023
31. Rana K, Haviland J, Garg S, Abou-Chakra J, Reid I, Suenderhauf N. Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning. In: 7th Annual Conference on Robot Learning. 2023
32. Zhu A, Martin L, Head A, Callison-Burch C. Calypso: Llms as dungeon master's assistants. In: Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment. 2023, 380–390
33. Wang Z, Cai S, Chen G, Liu A, Ma X, Liang Y. Describe, explain, plan and select: Interactive planning with large language models enables open-world multitask agents. arXiv preprint arXiv:2302.01560, 2023
34. Lin J, Zhao H, Zhang A, Wu Y, Ping H, Chen Q. Agentsims: An open-source sandbox for large language model evaluation. arXiv preprint arXiv:2308.04026, 2023
35. Liang X, Wang B, Huang H, Wu S, Wu P, Lu L, Ma Z, Li Z. Unleashing infinite-length input capacity for large-scale language models with self-controlled memory system. arXiv preprint arXiv:2304.13343, 2023
36. Ng Y, Miyashita D, Hoshi Y, Morioka Y, Torii O, Kodama T, Deguchi J. Simplyretrieve: A private and lightweight retrieval-centric generative ai tool. arXiv preprint arXiv:2308.03983, 2023
37. Huang Z, Gutierrez S, Kamana H, MacNeil S. Memory sandbox: Transparent and interactive memory management for conversational agents. In: Adjunct Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology. 2023, 1–3
38. Wang G, Xie Y, Jiang Y, Mandlekar A, Xiao C, Zhu Y, Fan L, Anandkumar A. Voyager: An open-ended embodied agent with large language models. arXiv preprint arXiv:2305.16291, 2023
39. Zhong W, Guo L, Gao Q, Wang Y. Memorybank: Enhancing large language models with long-term memory. arXiv preprint arXiv:2305.10250, 2023
40. Hu C, Fu J, Du C, Luo S, Zhao J, Zhao H. Chatdb: Augmenting llms with databases as their symbolic memory. arXiv preprint arXiv:2306.03901, 2023
41. Modarressi A, Imani A, Fayyaz M, Schütze H. Ret-llm: Towards a general read-write memory for large language models. arXiv preprint arXiv:2305.14322, 2023
42. Schuurmans D. Memory augmented large language models are computationally universal. arXiv preprint arXiv:2301.04589, 2023
43. Zhao A, Huang D, Xu Q, Lin M, Liu Y J, Huang G. Expel: Llm agents are experiential learners. arXiv preprint arXiv:2308.10144, 2023
44. Huang W, Abbeel P, Pathak D, Mordatch I. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In: International Conference on Machine Learning. 2022, 9118–9147
45. Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi E, Le Q V, Zhou D, others . Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems, 2022, 35: 24824–24837
46. Kojima T, Gu S S, Reid M, Matsuo Y, Iwasawa Y. Large language models are zero-shot reasoners. Advances in neural information processing systems, 2022, 35: 22199–22213
47. Raman S S, Cohen V, Rosen E, Idrees I, Paulius D, Tellex S. Planning with large language models via corrective re-prompting. In: NeurIPS 2022 Foundation Models for Decision Making Workshop. 2022
48. Xu B, Peng Z, Lei B, Mukherjee S, Liu Y, Xu D. Rewoo: Decoupling reasoning from observations for efficient augmented language models. arXiv preprint arXiv:2305.18323, 2023
49. Lin B Y, Fu Y, Yang K, Brahman F, Huang S, Bhagavatula C, Ammanabrolu P, Choi Y, Ren X. Swiftsage:

A generative agent with fast and slow thinking for complex interactive tasks. Advances in Neural Information Processing Systems, 2024, 36

50. Evans J S B, Stanovich K E. Dual-process theories of higher cognition: Advancing the debate. Perspectives on psychological science, 2013, 8(3): 223–241
51. Wang X, Wei J, Schuurmans D, Le Q, Chi E, Narang S, Chowdhery A, Zhou D. Self-consistency improves chain of thought reasoning in language models. arXiv preprint arXiv:2203.11171, 2022
52. Yao S, Yu D, Zhao J, Shafran I, Griffiths T, Cao Y, Narasimhan K. Tree of thoughts: Deliberate problem solving with large language models. Advances in Neural Information Processing Systems, 2024, 36
53. Wang Y, Jiang Z, Chen Z, Yang F, Zhou Y, Cho E, Fan X, Huang X, Lu Y, Yang Y. Recmind: Large language model powered agent for recommendation. arXiv preprint arXiv:2308.14296, 2023
54. Besta M, Blach N, Kubicek A, Gerstenberger R, Gianinazzi L, Gajda J, Lehmann T, Podstawski M, Niewiadomski H, Nyczyk P, others . Graph of thoughts: Solving elaborate problems with large language models. arXiv preprint arXiv:2308.09687, 2023
55. Sel B, Al-Tawaha A, Khattar V, Wang L, Jia R, Jin M. Algorithm of thoughts: Enhancing exploration of ideas in large language models. arXiv preprint arXiv:2308.10379, 2023
56. Gramopadhye M, Szafir D. Generating executable action plans with environmentally-aware language models. In: 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). 2023, 3568–3575
57. Hao S, Gu Y, Ma H, Hong J J, Wang Z, Wang D Z, Hu Z. Reasoning with language model is planning with world model. arXiv preprint arXiv:2305.14992, 2023
58. Liu B, Jiang Y, Zhang X, Liu Q, Zhang S, Biswas J, Stone P. LLM+P: Empowering large language models with optimal planning proficiency. arXiv preprint arXiv:2304.11477, 2023
59. Dagan G, Keller F, Lascarides A. Dynamic planning with a llm. arXiv preprint arXiv:2308.06391, 2023
60. Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, Cao Y. React: Synergizing reasoning and acting in language models. In: The Twelfth International Conference on Learning Representations. 2023
61. Song C H, Wu J, Washington C, Sadler B M, Chao W L, Su Y. Llm-planner: Few-shot grounded planning for embodied agents with large language models. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. 2023, 2998–3009
62. Huang W, Xia F, Xiao T, Chan H, Liang J, Florence P, Zeng A, Tompson J, Mordatch I, Chebotar Y, others . Inner monologue: Embodied reasoning through planning with language models. arXiv preprint arXiv:2207.05608, 2022
63. Madaan A, Tandon N, Gupta P, Hallinan S, Gao L, Wiegreffe S, Alon U, Dziri N, Prabhumoye S, Yang Y, others . Self-refine: Iterative refinement with self-feedback. Advances in Neural Information Processing Systems, 2024, 36
64. Miao N, Teh Y W, Rainforth T. Selfcheck: Using llms to zero-shot check their own step-by-step reasoning. In: The Twelfth International Conference on Learning Representations. 2023
65. Chen P L, Chang C S. Interact: Exploring the potentials of chatgpt as a cooperative agent. arXiv preprint arXiv:2308.01552, 2023
66. Chen Z, Zhou K, Zhang B, Gong Z, Zhao W X, Wen J R. Chatcot: Tool-augmented chain-of-thought reasoning on\\chat-based large language models. arXiv preprint arXiv:2305.14323, 2023
67. Nakano R, Hilton J, Balaji S, Wu J, Ouyang L, Kim C, Hesse C, Jain S, Kosaraju V, Saunders W, others . Webgpt: Browser-assisted question-answering with human feedback. arXiv preprint arXiv:2112.09332, 2021
68. Ruan J, Chen Y, Zhang B, Xu Z, Bao T, Du G, Shi S, Mao H, Zeng X, Zhao R. TPTU: Task planning and tool usage of large language model-based AI agents. arXiv preprint arXiv:2308.03427, 2023
69. Patil S G, Zhang T, Wang X, Gonzalez J E. Gorilla: Large language model connected with massive apis. arXiv preprint arXiv:2305.15334, 2023
70. Li M, Song F, Yu B, Yu H, Li Z, Huang F, Li Y. Api-bank: A benchmark for tool-augmented llms. arXiv preprint arXiv:2304.08244, 2023
71. Song Y, Xiong W, Zhu D, Li C, Wang K, Tian Y, Li S. Restgpt: Connecting large language models with real-world applications via restful apis. arXiv preprint arXiv:2306.06624, 2023
72. Liang Y, Wu C, Song T, Wu W, Xia Y, Liu Y, Ou Y, Lu S, Ji L, Mao S, others . Taskmatrix. ai: Completing tasks by connecting foundation models with millions of apis. Intelligent Computing, 2024, 3: 0063
73. Karpas E, Abend O, Belinkov Y, Lenz B, Lieber O, Ratner N, Shoham Y, Bata H, Levine Y, Leyton-Brown K, others . Mrkl systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. arXiv preprint arXiv:2205.00445, 2022
74. Ge Y, Hua W, Mei K, Tan J, Xu S, Li Z, Zhang Y, others . Openagi: When llm meets domain experts. Advances

in Neural Information Processing Systems, 2024, 36
75. Surís D, Menon S, Vondrick C. Vipergpt: Visual inference via python execution for reasoning. arXiv preprint arXiv:2303.08128, 2023
76. Bran A M, Cox S, White A D, Schwaller P. Chemcrow: Augmenting large-language models with chemistry tools. arXiv preprint arXiv:2304.05376, 2023
77. Yang Z, Li L, Wang J, Lin K, Azarnasab E, Ahmed F, Liu Z, Liu C, Zeng M, Wang L. Mm-react: Prompting chatgpt for multimodal reasoning and action. arXiv preprint arXiv:2303.11381, 2023
78. Gao C, Lan X, Lu Z, Mao J, Piao J, Wang H, Jin D, Li Y. S3: Social-network simulation system with large language model-empowered agents. arXiv preprint arXiv:2307.14984, 2023
79. Ahn M, Brohan A, Brown N, Chebotar Y, Cortes O, David B, Finn C, Fu C, Gopalakrishnan K, Hausman K, others . Do as i can, not as i say: Grounding language in robotic affordances. arXiv preprint arXiv:2204.01691, 2022
80. Park J S, Popowski L, Cai C, Morris M R, Liang P, Bernstein M S. Social simulacra: Creating populated prototypes for social computing systems. In: Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology. 2022, 1–18
81. Li G, Hammoud H A A K, Itani H, Khizbullin D, Ghanem B. Camel: Communicative agents for" mind" exploration of large scale language model society. arXiv preprint arXiv:2303.17760, 2023
82. al. e T. Auto-GPT. https://github.com/Significant-Gravitas/Auto-GPT, 2023
83. Liu R, Yang R, Jia C, Zhang G, Zhou D, Dai A M, Yang D, Vosoughi S. Training socially aligned language models in simulated human society. arXiv preprint arXiv:2305.16960, 2023
84. Chen L, Wang L, Dong H, Du Y, Yan J, Yang F, Li S, Zhao P, Qin S, Rajmohan S, others . Introspective tips: Large language model for in-context decision making. arXiv preprint arXiv:2305.11598, 2023
85. Liu H, Sferrazza C, Abbeel P. Chain of hindsight aligns language models with feedback. In: The Twelfth International Conference on Learning Representations. 2023
86. Yao S, Chen H, Yang J, Narasimhan K. Webshop: Towards scalable real-world web interaction with grounded language agents. Advances in Neural Information Processing Systems, 2022, 35: 20744–20757
87. Dan Y, Lei Z, Gu Y, Li Y, Yin J, Lin J, Ye L, Tie Z, Zhou Y, Wang Y, others . Educhat: A large-scale language model-based chatbot system for intelligent education. arXiv preprint arXiv:2308.02773, 2023
88. Deng X, Gu Y, Zheng B, Chen S, Stevens S, Wang B, Sun H, Su Y. Mind2web: Towards a generalist agent for the web. Advances in Neural Information Processing Systems, 2024, 36
89. Sun R, Arik S O, Nakhost H, Dai H, Sinha R, Yin P, Pfister T. Sql-palm: Improved large language modeladaptation for text-to-sql. arXiv preprint arXiv:2306.00739, 2023
90. Yao W, Heinecke S, Niebles J C, Liu Z, Feng Y, Xue L, Murthy R, Chen Z, Zhang J, Arpit D, Xu R, Mui P, Wang H, Xiong C, Savarese S. Retroformer: Retrospective large language agents with policy gradient optimization, 2023
91. Shu Y, Gu H, Zhang P, Zhang H, Lu T, Li D, Gu N. Rah! recsys-assistant-human: A human-central recommendation framework with large language models. arXiv preprint arXiv:2308.09904, 2023
92. Mandi Z, Jain S, Song S. Roco: Dialectic multi-robot collaboration with large language models. arXiv preprint arXiv:2307.04738, 2023
93. Zhang C, Liu L, Wang J, Wang C, Sun X, Wang H, Cai M. Prefer: Prompt ensemble learning via feedback-reflect-refine. arXiv preprint arXiv:2308.12033, 2023
94. Du Y, Li S, Torralba A, Tenenbaum J B, Mordatch I. Improving factuality and reasoning in language models through multiagent debate. arXiv preprint arXiv:2305.14325, 2023
95. Yang Z, Liu J, Han Y, Chen X, Huang Z, Fu B, Yu G. Appagent: Multimodal agents as smartphone users. arXiv preprint arXiv:2312.13771, 2023
96. Madaan A, Tandon N, Clark P, Yang Y. Memory-assisted prompt editing to improve GPT-3 after deployment. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. 2022
97. Colas C, Teodorescu L, Oudeyer P Y, Yuan X, Côté M A. Augmenting autotelic agents with large language models. arXiv preprint arXiv:2305.12487, 2023
98. Nascimento N, Alencar P, Cowan D. Self-adaptive large language model (llm)-based multiagent systems. In: 2023 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C). 2023, 104–109
99. Saha S, Hase P, Bansal M. Can language models teach weaker agents? teacher explanations improve students via theory of mind. arXiv preprint arXiv:2306.09299, 2023
100. Zhuge M, Liu H, Faccio F, Ashley D R, Csordás R, Gopalakrishnan A, Hamdi A, Hammoud H A A K, Herrmann V, Irie K, others . Mindstorms in natural language-based societies of mind. arXiv preprint

arXiv:2305.17066, 2023

101. Aher G V, Arriaga R I, Kalai A T. Using large language models to simulate multiple humans and replicate human subject studies. In: International Conference on Machine Learning. 2023, 337–371
102. Akata E, Schulz L, Coda-Forno J, Oh S J, Bethge M, Schulz E. Playing repeated games with large language models. arXiv preprint arXiv:2305.16867, 2023
103. Ma Z, Mei Y, Su Z. Understanding the benefits and challenges of using large language model-based conversational agents for mental well-being support. In: AMIA Annual Symposium Proceedings. 2023, 1105
104. Ziems C, Held W, Shaikh O, Chen J, Zhang Z, Yang D. Can large language models transform computational social science? arXiv preprint arXiv:2305.03514, 2023
105. Horton J J. Large language models as simulated economic agents: What can we learn from homo silicus? Technical report, National Bureau of Economic Research, 2023
106. Li S, Yang J, Zhao K. Are you in a masquerade? exploring the behavior and impact of large language model driven social bots in online social networks. arXiv preprint arXiv:2307.10337, 2023
107. Li C, Su X, Fan C, Han H, Xue C, Zheng C. Quantifying the impact of large language models on collective opinion dynamics. arXiv preprint arXiv:2308.03313, 2023
108. Kovač G, Portelas R, Dominey P F, Oudeyer P Y. The socialai school: Insights from developmental psychology towards artificial socio-cultural agents. arXiv preprint arXiv:2307.07871, 2023
109. Williams R, Hosseinichimeh N, Majumdar A, Ghaffarzadegan N. Epidemic modeling with generative agents. arXiv preprint arXiv:2307.04986, 2023
110. Jinxin S, Jiabao Z, Yilei W, Xingjiao W, Jiawen L, Liang H. Cgmi: Configurable general multi-agent interaction framework. arXiv preprint arXiv:2308.12503, 2023
111. Cui J, Li Z, Yan Y, Chen B, Yuan L. Chatlaw: Open-source legal large language model with integrated external knowledge bases. arXiv preprint arXiv:2306.16092, 2023
112. Hamilton S. Blind judgement: Agent-based supreme court modelling with gpt. arXiv preprint arXiv:2301.05327, 2023
113. Bail C A. Can generative ai improve social science? 2023
114. Boiko D A, MacKnight R, Gomes G. Emergent autonomous scientific research capabilities of large language models. arXiv preprint arXiv:2304.05332, 2023
115. Kang Y, Kim J. Chatmof: An autonomous ai system for predicting and generating metal-organic frameworks. arXiv preprint arXiv:2308.01423, 2023
116. Swan M, Kido T, Roland E, Santos R P d. Math agents: Computational infrastructure, mathematical embedding, and genomics. arXiv preprint arXiv:2307.02502, 2023
117. Drori I, Zhang S, Shuttleworth R, Tang L, Lu A, Ke E, Liu K, Chen L, Tran S, Cheng N, others . A neural network solves, explains, and generates university math problems by program synthesis and few-shot learning at human level. Proceedings of the National Academy of Sciences, 2022, 119(32): e2123433119
118. Chen M, Tworek J, Jun H, Yuan Q, Pinto H P d O, Kaplan J, Edwards H, Burda Y, Joseph N, Brockman G, others . Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374, 2021
119. Liffiton M, Sheese B E, Savelka J, Denny P. Codehelp: Using large language models with guardrails for scalable support in programming classes. In: Proceedings of the 23rd Koli Calling International Conference on Computing Education Research. 2023, 1–11
120. Matelsky J K, Parodi F, Liu T, Lange R D, Kording K P. A large language model-assisted education tool to provide feedback on open-ended responses. arXiv preprint arXiv:2308.02439, 2023
121. Grossmann I, Feinberg M, Parker D C, Christakis N A, Tetlock P E, Cunningham W A. Ai and the transformation of social science research. Science, 2023, 380(6650): 1108–1109
122. Zhou X, Li G, Liu Z. Llm as dba. arXiv preprint arXiv:2308.05481, 2023
123. He Z, Wu H, Zhang X, Yao X, Zheng S, Zheng H, Yu B. Chateda: A large language model powered autonomous agent for eda. In: 2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD). 2023, 1–6
124. Huang X, Lian J, Lei Y, Yao J, Lian D, Xie X. Recommender ai agent: Integrating large language models for interactive recommendations. arXiv preprint arXiv:2308.16505, 2023
125. Deng G, Liu Y, Mayoral-Vilches V, Liu P, Li Y, Xu Y, Zhang T, Liu Y, Pinzger M, Rass S. Pentestgpt: An llm-empowered automatic penetration testing tool. arXiv preprint arXiv:2308.06782, 2023
126. al. e S. Smolmodels. https://github.com/smol-ai/developer, 2023
127. al. e M U. DemoGPT. https://github.com/melih-unsal/DemoGPT, 2023
128. al. e A O. GPT engineer. https://github.com/AntonOsika/gpt-engineer, 2023
129. Xia Y, Shenoy M, Jazdi N, Weyrich M. Towards autonomous system: flexible modular production system

enhanced with large language model agents. arXiv preprint arXiv:2304.14721, 2023
130. Ogundare O, Madasu S, Wiggins N. Industrial engineering with large language models: A case study of chatgpt's performance on oil & gas problems. arXiv preprint arXiv:2304.14354, 2023
131. Zhang C, Yang K, Hu S, Wang Z, Li G, Sun Y, Zhang C, Zhang Z, Liu A, Zhu S C, others . Proagent: Building proactive cooperative ai with large language models. arXiv preprint arXiv:2308.11339, 2023
132. Hu B, Zhao C, others . Enabling intelligent interactions between an agent and an llm: A reinforcement learning approach. arXiv:2306.03604, 2023
133. Wu Y, Min S Y, Bisk Y, Salakhutdinov R, Azaria A, Li Y, Mitchell T, Prabhumoye S. Plan, eliminate, and track–language models are good teachers for embodied agents. arXiv preprint arXiv:2305.02412, 2023
134. Zhang D, Chen L, Zhang S, Xu H, Zhao Z, Yu K. Large language models are semi-parametric reinforcement learning agents. Advances in Neural Information Processing Systems, 2024, 36
135. Di Palo N, Byravan A, Hasenclever L, Wulfmeier M, Heess N, Riedmiller M. Towards a unified agent with foundation models. In: Workshop on Reincarnating Reinforcement Learning at ICLR 2023. 2023
136. Wu J, Antonova R, Kan A, Lepert M, Zeng A, Song S, Bohg J, Rusinkiewicz S, Funkhouser T. Tidybot: Personalized robot assistance with large language models. arXiv preprint arXiv:2305.05658, 2023
137. Wu Z, Wang Z, Xu X, Lu J, Yan H. Embodied task planning with large language models. arXiv preprint arXiv:2307.01848, 2023
138. Dasgupta I, Kaeser-Chen C, Marino K, Ahuja A, Babayan S, Hill F, Fergus R. Collaborating with language models for embodied reasoning. arXiv preprint arXiv:2302.00763, 2023
139. Nottingham K, Ammanabrolu P, Suhr A, Choi Y, Hajishirzi H, Singh S, Fox R. Do embodied agents dream of pixelated sheep?: Embodied decision making using language guided world modelling. In: Workshop on Reincarnating Reinforcement Learning at ICLR 2023. 2023
140. Zhou W, Peng X, Riedl M. Dialogue shaping: Empowering agents through npc interaction. arXiv preprint arXiv:2307.15833, 2023
141. Li H, Hao Y, Zhai Y, Qian Z. The hitchhiker's guide to program analysis: A journey with large language models. arXiv preprint arXiv:2308.00245, 2023
142. al. e R. AgentGPT. https://github.com/reworkd/AgentGPT, 2023
143. al. e E. Ai-legion. https://github.com/eumemic/ai-legion, 2023
144. al. e J X. Agixt. https://github.com/Josh-XT/AGiXT, 2023
145. al. e C. Xlang. https://github.com/xlang-ai/xlang, 2023
146. al. e N. Babyagi. https://github.com/yoheinakajima, 2023
147. Chase H. langchain. https://docs.langchain.com/docs/, 2023
148. al. e A M. WorkGPT. https://github.com/team-openpm/workgpt, 2023
149. al. e F R. LoopGPT. https://github.com/farizrahman4u/loopgpt, 2023
150. al. e A E. GPT-researcher. https://github.com/assafelovic/gpt-researcher, 2023
151. Qin Y, Hu S, Lin Y, Chen W, Ding N, Cui G, Zeng Z, Huang Y, Xiao C, Han C, others . Tool learning with foundation models. arXiv preprint arXiv:2304.08354, 2023
152. Face H. transformers-agent. https://huggingface.co/docs/transformers/transformers_agents, 2023
153. al. e E. Miniagi. https://github.com/muellerberndt/mini-agi, 2023
154. al. e T. Superagi. https://github.com/TransformerOptimus/SuperAGI, 2023
155. Wu Q, Bansal G, Zhang J, Wu Y, Zhang S, Zhu E, Li B, Jiang L, Zhang X, Wang C. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. arXiv preprint arXiv:2308.08155, 2023
156. Chen W, Su Y, Zuo J, Yang C, Yuan C, Qian C, Chan C M, Qin Y, Lu Y, Xie R, others . Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents. arXiv preprint arXiv:2308.10848, 2023
157. Lee M, Srivastava M, Hardy A, Thickstun J, Durmus E, Paranjape A, Gerard-Ursin I, Li X L, Ladhak F, Rong F, others . Evaluating human-language model interaction. arXiv preprint arXiv:2212.09746, 2022
158. Chan C M, Chen W, Su Y, Yu J, Xue W, Zhang S, Fu J, Liu Z. Chateval: Towards better llm-based evaluators through multi-agent debate. arXiv preprint arXiv:2308.07201, 2023
159. Kang S, Yoon J, Yoo S. Large language models are few-shot testers: Exploring llm-based general bug reproduction. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). 2023, 2312–2323
160. Jalil S, Rafi S, LaToza T D, Moran K, Lam W. Chatgpt and software testing education: Promises & perils. In: 2023 IEEE International Conference on Software Test-

ing, Verification and Validation Workshops (ICSTW). 2023, 4130–4137

161. Mehta N, Teruel M, Sanz P F, Deng X, Awadallah A H, Kiseleva J. Improving grounded language understanding in a collaborative environment by interacting with agents through help feedback. arXiv preprint arXiv:2304.10750, 2023
162. Chen A, Phang J, Parrish A, Padmakumar V, Zhao C, Bowman S R, Cho K. Two failures of self-consistency in the multi-step reasoning of llms. arXiv preprint arXiv:2305.14279, 2023
163. Choi M, Pei J, Kumar S, Shu C, Jurgens D. Do llms understand social knowledge? evaluating the sociability of large language models with socket benchmark. arXiv preprint arXiv:2305.14938, 2023
164. Zhang D, Chen L, Zhao Z, Cao R, Yu K. Mobile-env: An evaluation platform and benchmark for interactive agents in llm era. arXiv preprint arXiv:2305.08144, 2023
165. Chalamalasetti K, Götze J, Hakimov S, Madureira B, Sadler P, Schlangen D. clembench: Using game play to evaluate chat-optimized language models as conversational agents. arXiv preprint arXiv:2305.13455, 2023
166. Lin J, Tomlin N, Andreas J, Eisner J. Decision-oriented dialogue for human-ai collaboration. arXiv preprint arXiv:2305.20076, 2023
167. Feldt R, Kang S, Yoon J, Yoo S. Towards autonomous testing agents via conversational large language models. arXiv preprint arXiv:2306.05152, 2023
168. Liang Y, Zhu L, Yang Y. Tachikuma: Understading complex interactions with multi-character and novel objects by large language models. arXiv preprint arXiv:2307.12573, 2023
169. Liu X, Yu H, Zhang H, Xu Y, Lei X, Lai H, Gu Y, Ding H, Men K, Yang K, others . Agentbench: Evaluating llms as agents. arXiv preprint arXiv:2308.03688, 2023
170. Liu Z, Yao W, Zhang J, Xue L, Heinecke S, Murthy R, Feng Y, Chen Z, Niebles J C, Arpit D, others . Bolaa: Benchmarking and orchestrating llm-augmented autonomous agents. arXiv preprint arXiv:2308.05960, 2023
171. Xu B, Liu X, Shen H, Han Z, Li Y, Yue M, Peng Z, Liu Y, Yao Z, Xu D. Gentopia. ai: A collaborative platform for tool-augmented llms. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. 2023, 237–245
172. Huang J t, Lam M H, Li E J, Ren S, Wang W, Jiao W, Tu Z, Lyu M R. Emotionally numb or empathetic? evaluating how llms feel using emotionbench. arXiv preprint arXiv:2308.03656, 2023
173. Zhou S, Xu F F, Zhu H, Zhou X, Lo R, Sridhar A, Cheng X, Bisk Y, Fried D, Alon U, others . Webarena: A realistic web environment for building autonomous agents. arXiv preprint arXiv:2307.13854, 2023
174. Banerjee D, Singh P, Avadhanam A, Srivastava S. Benchmarking llm powered chatbots: methods and metrics. arXiv preprint arXiv:2308.04624, 2023
175. Zhao W X, Zhou K, Li J, Tang T, Wang X, Hou Y, Min Y, Zhang B, Zhang J, Dong Z, others . A survey of large language models. arXiv preprint arXiv:2303.18223, 2023
176. Yang J, Jin H, Tang R, Han X, Feng Q, Jiang H, Zhong S, Yin B, Hu X. Harnessing the power of llms in practice: A survey on chatgpt and beyond. ACM Transactions on Knowledge Discovery from Data, 2023
177. Wang Y, Zhong W, Li L, Mi F, Zeng X, Huang W, Shang L, Jiang X, Liu Q. Aligning large language models with human: A survey. arXiv preprint arXiv:2307.12966, 2023
178. Huang J, Chang K C C. Towards reasoning in large language models: A survey. arXiv preprint arXiv:2212.10403, 2022
179. Mialon G, Dessì R, Lomeli M, Nalmpantis C, Pasunuru R, Raileanu R, Rozière B, Schick T, Dwivedi-Yu J, Celikyilmaz A, others . Augmented language models: a survey. arXiv preprint arXiv:2302.07842, 2023
180. Chang Y, Wang X, Wang J, Wu Y, Yang L, Zhu K, Chen H, Yi X, Wang C, Wang Y, others . A survey on evaluation of large language models. ACM Transactions on Intelligent Systems and Technology, 2023
181. Chang T A, Bergen B K. Language model behavior: A comprehensive survey. Computational Linguistics, 2024, 1–58
182. Li C, Wang J, Zhu K, Zhang Y, Hou W, Lian J, Xie X. Emotionprompt: Leveraging psychology for large language models enhancement via emotional stimulus. arXiv e-prints, 2023, arXiv–2307
183. Zhuo T Y, Li Z, Huang Y, Shiri F, Wang W, Haffari G, Li Y F. On robustness of prompt-based semantic parsing with large pre-trained language model: An empirical study on codex. arXiv preprint arXiv:2301.12868, 2023
184. Gekhman Z, Oved N, Keller O, Szpektor I, Reichart R. On the robustness of dialogue history representation in conversational question answering: a comprehensive study and a new prompt-based method. Transactions of the Association for Computational Linguistics, 2023, 11: 351–366
185. Ji Z, Lee N, Frieske R, Yu T, Su D, Xu Y, Ishii E, Bang Y J, Madotto A, Fung P. Survey of hallucination in

natural language generation. ACM Computing Surveys, 2023, 55(12): 1–38

Lei Wang is a Ph.D. candidate at Renmin University of China, Beijing. His research focuses on recommender systems and agent-based large language models.

Chen Ma is currently pursuing a Master's degree at Renmin University of China, Beijing, China. His research interests include recommender system, agent based on large language model.

Xueyang Feng is currently studying for a Ph.D. degree at Renmin University of China, Beijing, China. His research interests include recommender system, agent based on large language model.

Zeyu Zhang is currently pursuing a Master's degree at Renmin University of China, Beijing, China. His research interests include recommender system, causal inference, agent based on large language model.

Hao Yang is currently studying for a Ph.D. degree at Renmin University of China, Beijing, China. His research interests include recommender system, causal inference.

Jingsen Zhang is currently studying for a Ph.D. degree at Renmin University of China, Beijing, China. His research interests include recommender system.

Zhi-Yuan Chen is pursuing his Ph.D. in Gaoling school of Artificial Intelligence, Renmin University of China. His research mainly focuses on language model reasoning and agent based on large language model.

Jiakai Tang is currently pursuing a Master's degree at Renmin University of China, Beijing, China. His research interests include recommender system.

Xu Chen obtained his PhD degree from Tsinghua University, China. Before joining Renmin University of China, he was a postdoc researcher at University College London, UK. In the period from March to September of 2017, he was studying at Georgia Institute of Technology, USA, as a visiting scholar. His research mainly focuses on the recommender system, reinforcement learning and causal inference.

Yankai Lin received his B.E. and Ph.D. degrees from Tsinghua University in 2014 and 2019. After that, he worked as a senior researcher in Tencent WeChat, and joined Renmin University of China in 2022 as a tenure-track assistant professor. His main research interests are pretrained models and natural language processing.

Wayne Xin Zhao received his Ph.D. in Computer Science from Peking University in 2014. His research interests include data mining, natural language processing and information retrieval in general. The main goal is to study how to organize, analyze and mine user generated data for improving the service of real-world applications.

Zhewei Wei received his Ph.D. of Computer Science and Engineering from Hong Kong University of Science and Technology. He did postdoctoral research in Aarhus University from 2012 to 2014, and joined Renmin University of China in 2014.

Ji-Rong Wen is a full professor, the executive dean of Gaoling School of Artificial Intelligence, and the dean of School of Information at Renmin University of China. He has been working in the big data and AI areas for many years, and publishing extensively on prestigious international conferences and journals.